Final Project - Progress, April 28, 2026

Group 8

2026-04-28

Progress comments:

So far, the project has covered the main steps of analyzing the monthly U.S. unemployment rate (UNRATENSA) as a time series. The data were pulled from FRED, converted into a monthly time series, and cleaned by interpolating missing values.

Using STL decomposition, the data were split into trend, seasonal, and residual components. This showed a clear seasonal pattern and also revealed that the residuals still had noticeable autocorrelation. To model the structure, polynomial regression was used for the trend and harmonic regression for the seasonal component. Higher-degree polynomials improved the fit, but also introduced some instability, so there was a tradeoff to consider.

Since the regression models alone didn't fully capture the time dependence, autoregressive models were applied to the residuals. An AR(5) model was chosen as a reasonable balance between improving fit and avoiding overfitting. Combining the regression and AR components improved the overall model, although some dependence still remained.

Several forecasting methods were also tested, including seasonal naïve, simple exponential smoothing, Holt's method, and Holt-Winters. Holt-Winters worked best for this data because it accounts for both trend and seasonality.

Right now, the project is moving toward ARIMA modeling to better handle the remaining dependence in the data. That part is still in progress, including diagnostics and comparing forecasts to the other methods.

Initial data pull/clean

```r
#read the data from the fed website
library(readr)
url <- "https://fred.stlouisfed.org/graph/fredgraph.csv?id=UNRATENSA"
unratensa_data <- read_csv(url)

#convert to time series
unratensa_ts <- ts(unratensa_data$UNRATENSA, start = c(1948, 1), frequency = 12)
#interpolate the NA values
library(forecast)
unratensa_ts_interp <- na.interp(unratensa_ts)

#summary stats
summary(unratensa_ts_interp)
```

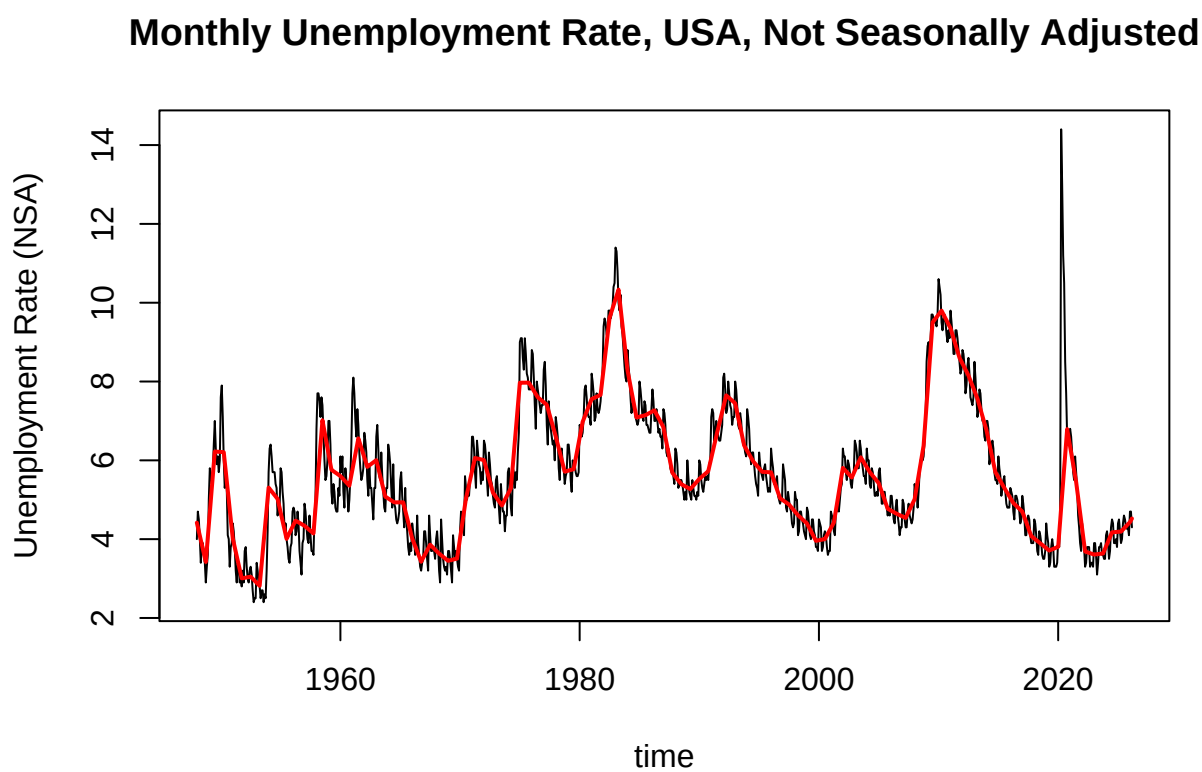
```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   2.400   4.400   5.400   5.662   6.700  14.400
```

```r
sd(unratensa_ts_interp)
```

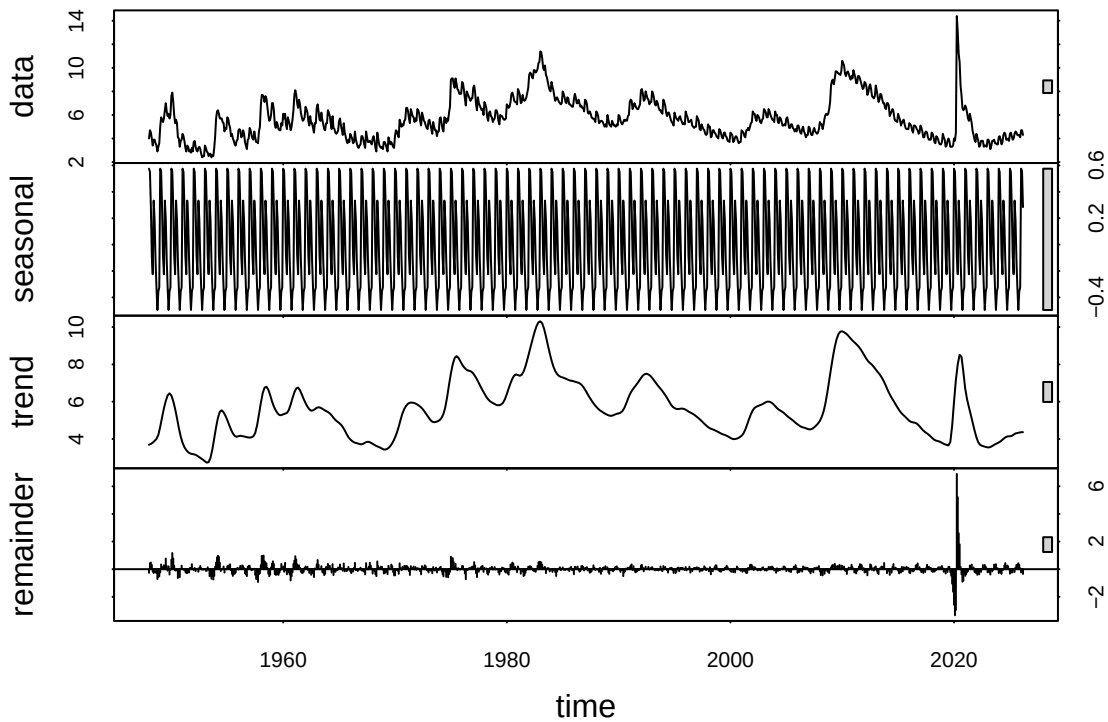
```
## [1] 1.744923
```

smoothing and stl

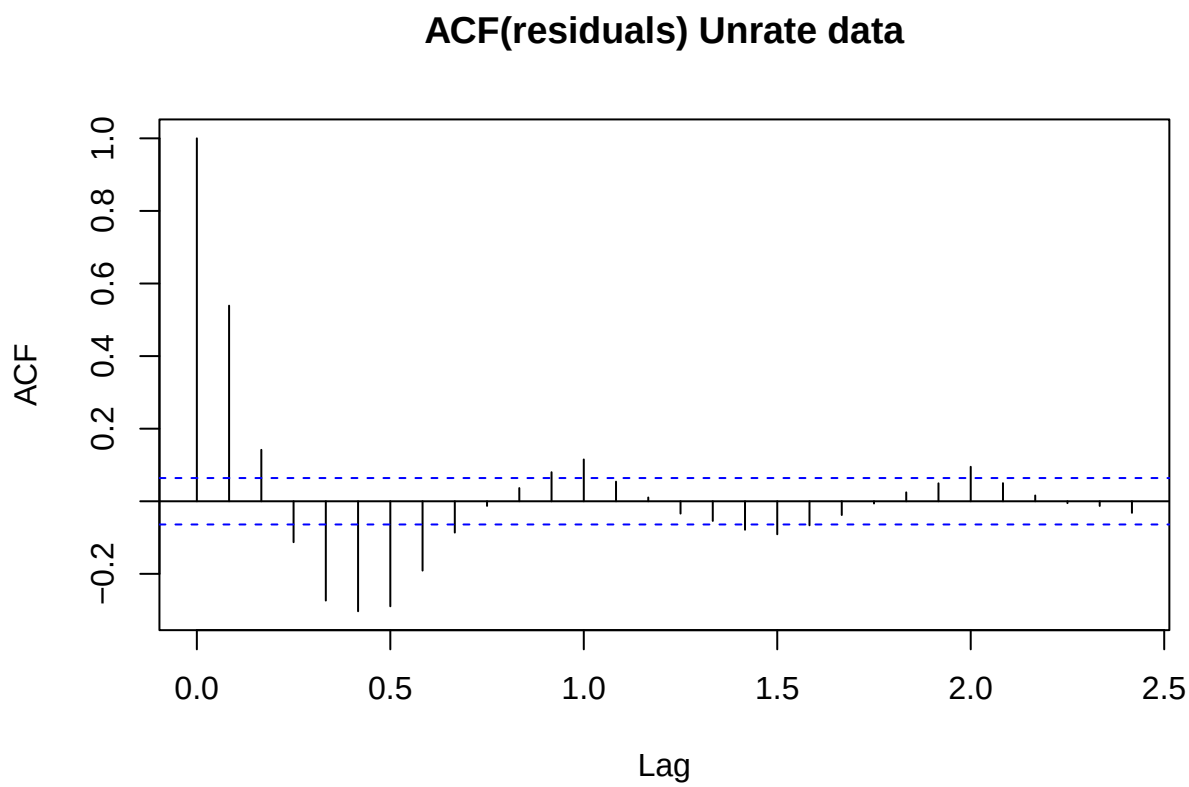
```r
#full data set and smoothing
plot(unratensa_ts_interp, ylab="Unemployment Rate (NSA)", xlab = "time", col = "black", main = "Monthly
lines(lowess(unratensa_ts_interp, f=0.01), lwd=2, col="red")
```




```r
#stl analysis
unratensa_stl <- stl(unratensa_ts_interp, s.window = "periodic")
plot(unratensa_stl)
```




```r
season <- unratensa_stl$time.series[,1]
trend <- unratensa_stl$time.series[,2]
res <- unratensa_stl$time.series[,3]

acf(res, main = "ACF(residuals) Unrate data")
```



Looking at the entire data set, we can see seasonality in the tiny spikes.

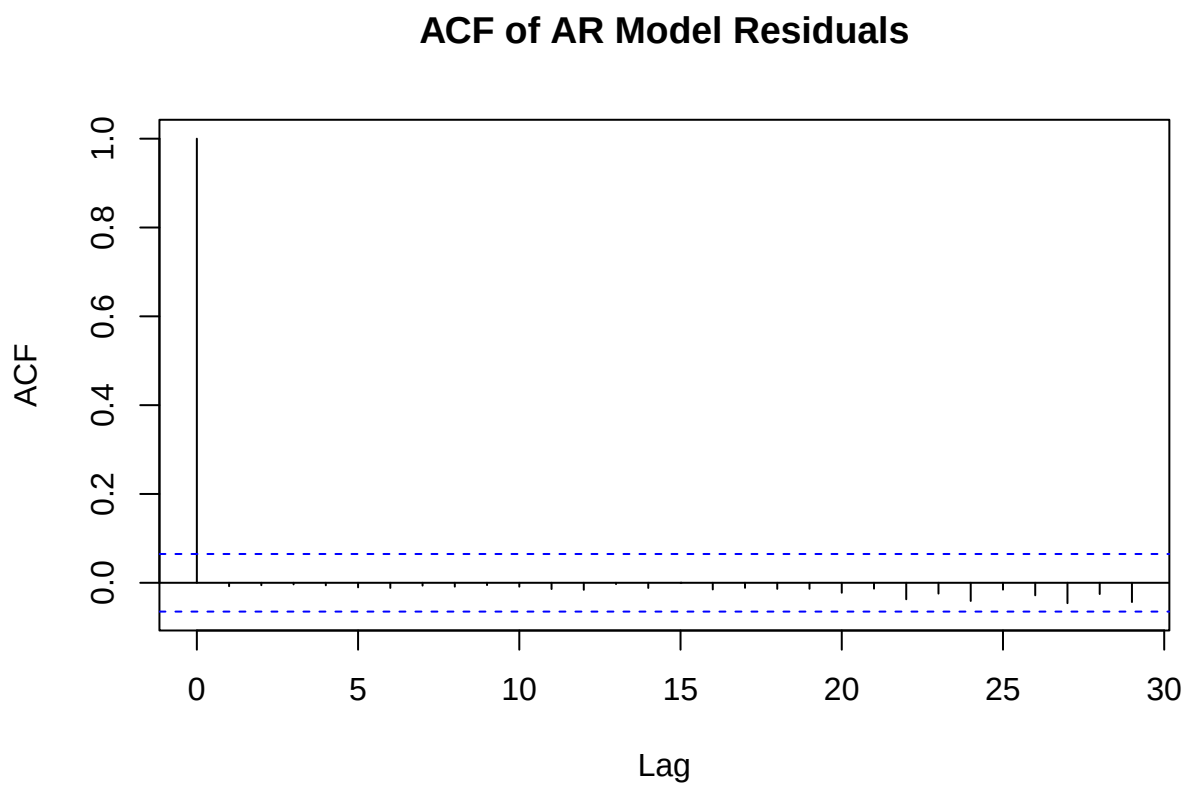
ACF reveals time dependence of the residuals, so we need to model the residuals.

Residual AR fit model

```
#get the order of the AR fit
fit_ar <- ar.yw(res, order.max = NULL)
fit_ar
```

```
##
## Call:
## ar.yw.default(x = res, order.max = NULL)
##
## Coefficients:
##       1        2        3        4        5        6        7        8
##  0.5390  -0.1878  -0.0777  -0.1762  -0.0508  -0.1600  -0.0535  -0.0702
##       9       10       11       12       13       14       15       16
## -0.0747  -0.0875  -0.0626   0.0171  -0.1280  -0.0235  -0.0851  -0.0188
##      17       18       19       20       21       22       23       24
## -0.0776  -0.0515  -0.0488  -0.0375  -0.0470  -0.0208  -0.0613   0.0722
##      25
## -0.0894
##
## Order selected 25  sigma^2 estimated as  0.1151
```

```r
# AR residuals
ar_resid <- ts(fit_ar$resid, start=start(unratensa_ts_interp), frequency=12)

# AR fitted values for the residual component
ar_fit <- res - ar_resid

# remove first p values when checking ACF because AR residuals begin with NA values
p <- fit_ar$order

acf(ar_resid[-c(1:p)],
    main = "ACF of AR Model Residuals")
```

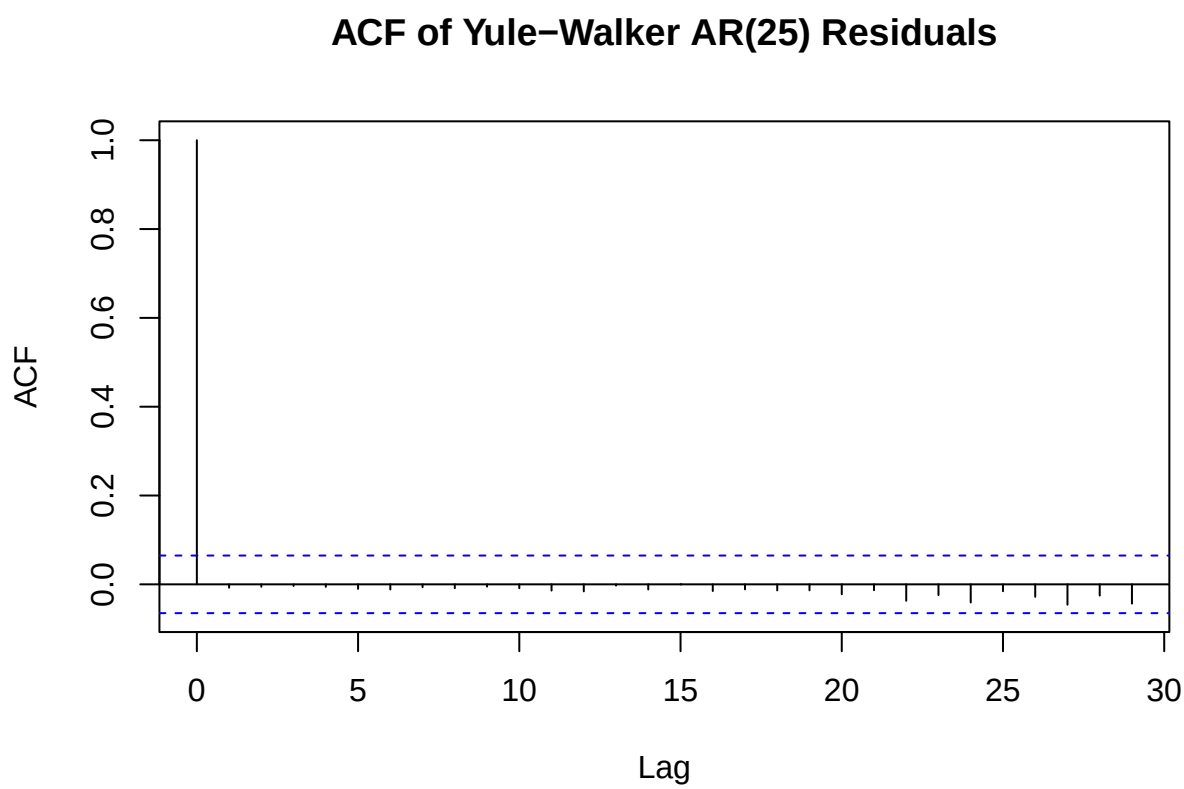



```r
#get the order of the AR fit using Yule-Walker
fit_ar <- ar.yw(res, order.max = NULL)
fit_ar
```

```
##
## Call:
## ar.yw.default(x = res, order.max = NULL)
##
## Coefficients:
##       1        2        3        4        5        6        7        8
##  0.5390  -0.1878  -0.0777  -0.1762  -0.0508  -0.1600  -0.0535  -0.0702
##       9       10       11       12       13       14       15       16
## -0.0747  -0.0875  -0.0626   0.0171  -0.1280  -0.0235  -0.0851  -0.0188
```

```
##      17       18       19       20       21       22       23       24
## -0.0776  -0.0515  -0.0488  -0.0375  -0.0470  -0.0208  -0.0613   0.0722
##      25
## -0.0894
##
## Order selected 25  sigma^2 estimated as  0.1151
```

```r
# AR residuals from Yule-Walker selected model
ar_resid <- ts(fit_ar$resid, start=start(unratensa_ts_interp), frequency=12)

# AR fitted values for the residual component
ar_fit <- res - ar_resid

# remove first p values when checking ACF because AR residuals begin with NA values
p <- fit_ar$order

acf(ar_resid[-c(1:p)],
    main = paste("ACF of Yule-Walker AR(", p, ") Residuals", sep = ""))
```

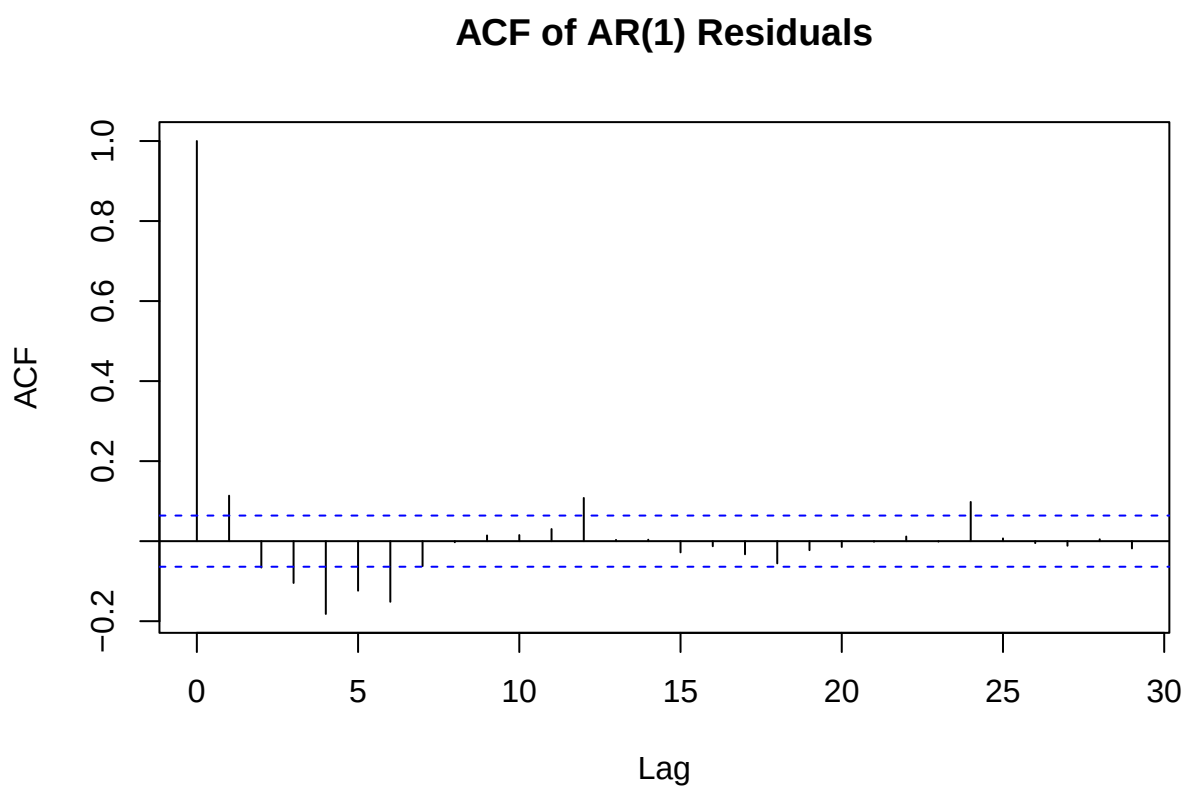



```r
#lower order AR
res_clean <- na.omit(as.numeric(res))

# Fit lower-order AR models
ar1 <- arima(res_clean, order = c(1, 0, 0), include.mean = TRUE)
ar2 <- arima(res_clean, order = c(2, 0, 0), include.mean = TRUE)
```

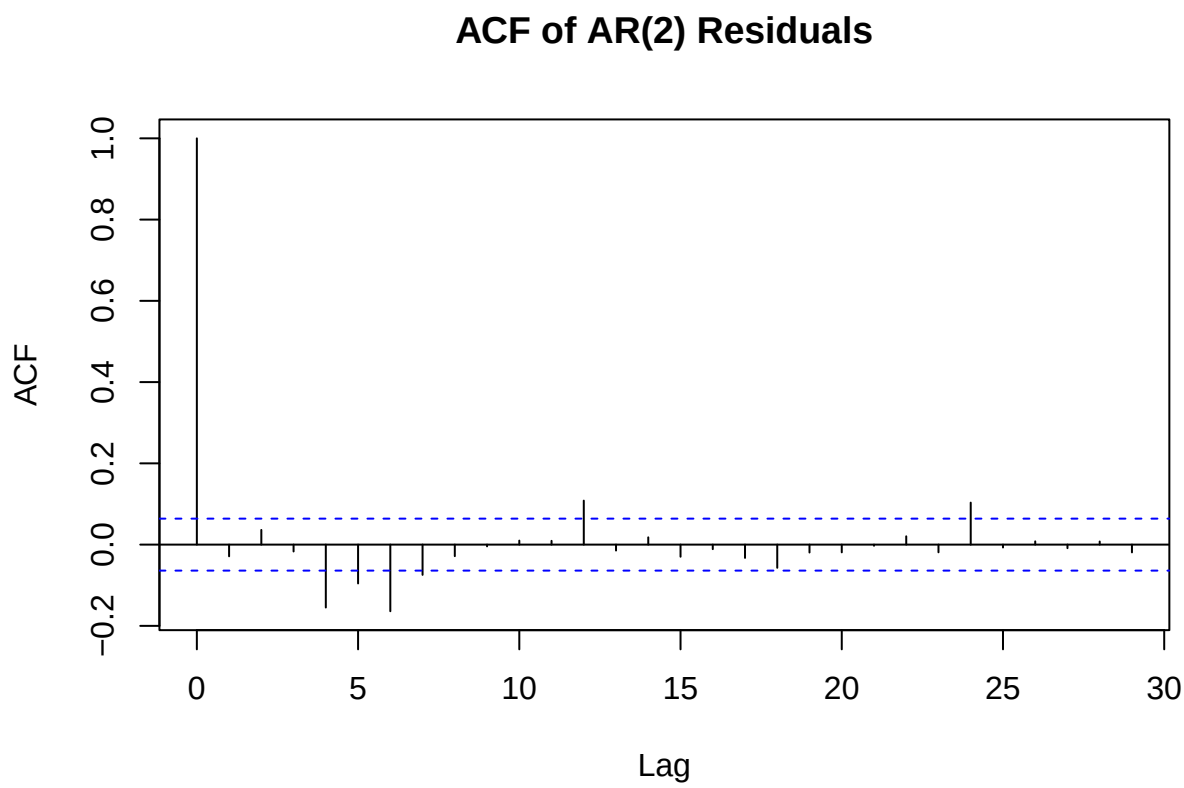
```r
ar3 <- arima(res_clean, order = c(3, 0, 0), include.mean = TRUE)
ar4 <- arima(res_clean, order = c(4, 0, 0), include.mean = TRUE)
ar5 <- arima(res_clean, order = c(5, 0, 0), include.mean = TRUE)

# Compare lower-order models using AIC
AIC(ar1, ar2, ar3, ar4, ar5)
```

```
##     df      AIC
## ar1  3 779.7643
## ar2  4 739.1523
## ar3  5 723.6950
## ar4  6 698.1354
## ar5  7 692.3861
```

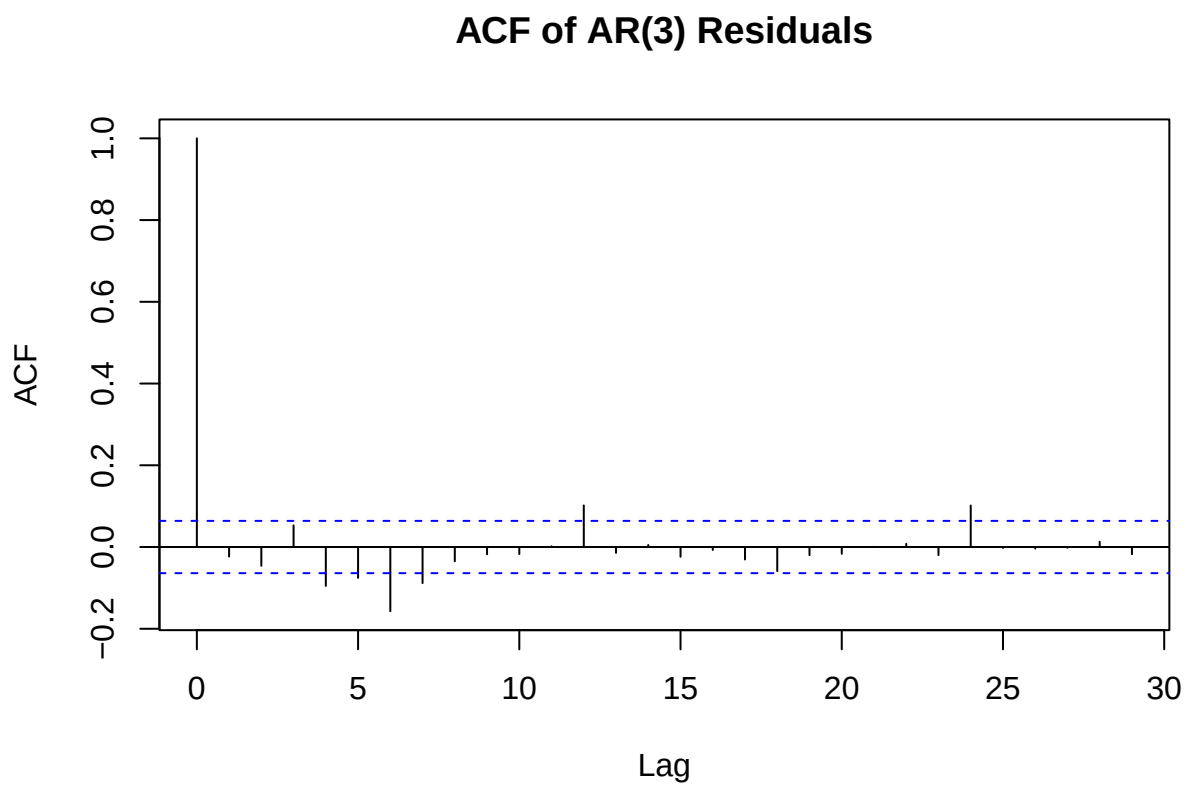
```r
# Check residual ACFs for lower-order AR models
acf(residuals(ar1), main = "ACF of AR(1) Residuals")
```



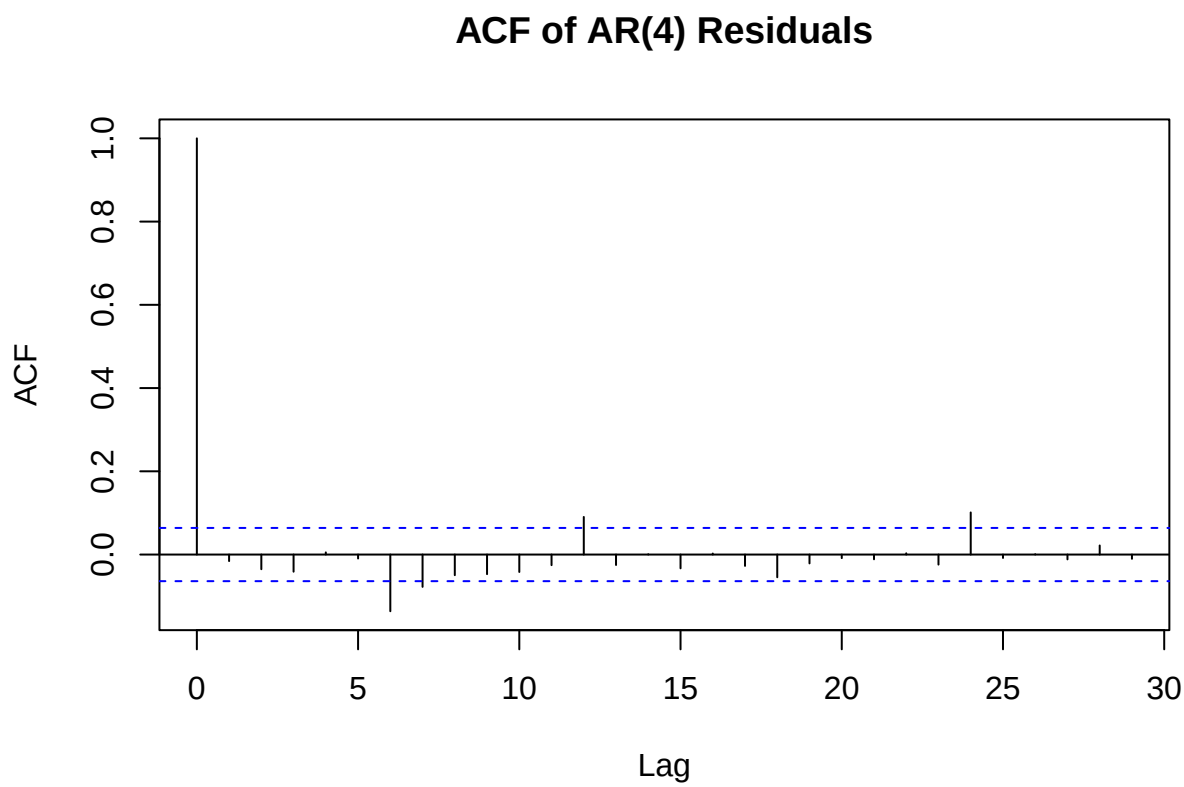

```r
acf(residuals(ar2), main = "ACF of AR(2) Residuals")
```



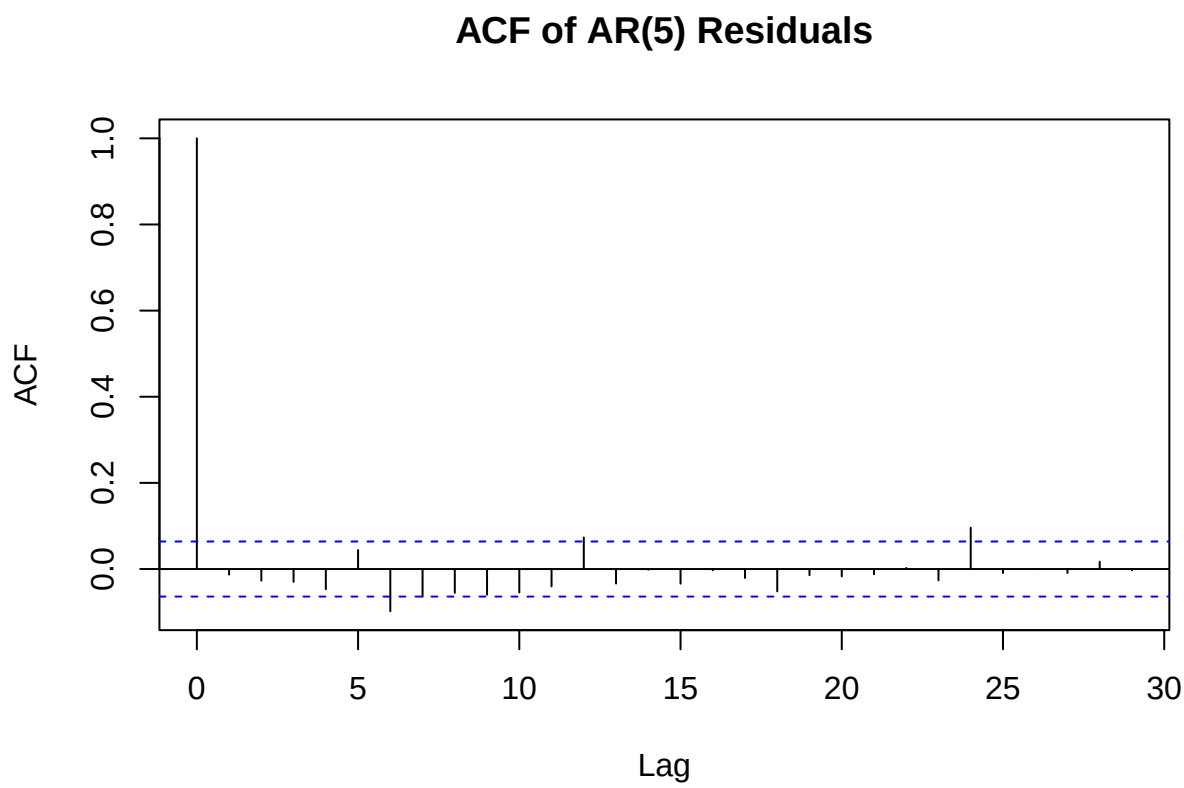

```r
acf(residuals(ar3), main = "ACF of AR(3) Residuals")
```




```r
acf(residuals(ar4), main = "ACF of AR(4) Residuals")
```




```r
acf(residuals(ar5), main = "ACF of AR(5) Residuals")
```




```r
AIC(ar1, ar2, ar3, ar4, ar5)
```

```
##     df      AIC
## ar1  3 779.7643
## ar2  4 739.1523
## ar3  5 723.6950
## ar4  6 698.1354
## ar5  7 692.3861
```

Yule-Walker method selected an AR(25) model, suggesting that the random shocks have a long-lasting effect on the data. Examining lower AR fits shows a consistent improvement in AIC, but with diminishing returns. We choose AR(5) as a compromise that improves fit, but does not overfit the model.

Trend model

```r
time <- 1:length(trend)
dat.trend <- data.frame(trend = trend, time = time)

# trig fit
fit_trig <- lm(trend ~
                 cos(2*pi*time/12) + sin(2*pi*time/12),
               data = dat.trend)
summary(fit_trig)
```

```
##
## Call:
## lm(formula = trend ~ cos(2 * pi * time/12) + sin(2 * pi * time/12),
##     data = dat.trend)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -2.9183 -1.2972 -0.1800  0.9667  4.6580
##
## Coefficients:
##                        Estimate Std. Error t value Pr(>|t|)
## (Intercept)            5.660154   0.052206 108.419   <2e-16 ***
## cos(2 * pi * time/12) -0.004646   0.073870  -0.063    0.950
## sin(2 * pi * time/12) -0.009344   0.073792  -0.127    0.899
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.6 on 936 degrees of freedom
## Multiple R-squared:  2.139e-05,  Adjusted R-squared:  -0.002115
## F-statistic: 0.01001 on 2 and 936 DF,  p-value: 0.99
```

```r
# polynomial fits
fit_poly3 <- lm(trend ~ time + I(time^2) + I(time^3), data = dat.trend)

fit_poly5 <- lm(trend ~ time + I(time^2) + I(time^3) +
                  I(time^4) + I(time^5), data = dat.trend)

fit_poly6 <- lm(trend ~ time + I(time^2) + I(time^3) +
                  I(time^4) + I(time^5) + I(time^6), data = dat.trend)

fit_poly7 <- lm(trend ~ time + I(time^2) + I(time^3) +
                  I(time^4) + I(time^5) + I(time^6) + I(time^7), data = dat.trend)

fit_poly8 <- lm(trend ~ time + I(time^2) + I(time^3) +
                  I(time^4) + I(time^5) + I(time^6) +
                  I(time^7) + I(time^8), data = dat.trend)

# summaries
summary(fit_poly3)
```

```
##
## Call:
## lm(formula = trend ~ time + I(time^2) + I(time^3), data = dat.trend)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -2.3700 -1.0547 -0.2808  0.8407  3.9877
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  3.971e+00  1.878e-01  21.149  < 2e-16 ***
## time         7.943e-03  1.729e-03   4.594 4.94e-06 ***
## I(time^2)   -3.613e-06  4.273e-06  -0.846    0.398
## I(time^3)   -4.726e-09  2.988e-09  -1.581    0.114
```

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.433 on 935 degrees of freedom
## Multiple R-squared:  0.1989, Adjusted R-squared:  0.1963
## F-statistic: 77.37 on 3 and 935 DF,  p-value: < 2.2e-16
```

```r
summary(fit_poly5)
```

```
##
## Call:
## lm(formula = trend ~ time + I(time^2) + I(time^3) + I(time^4) +
##     I(time^5), data = dat.trend)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -2.4769 -1.0720 -0.1943  0.8512  3.7451
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  5.292e+00  2.727e-01  19.408  < 2e-16 ***
## time        -3.385e-02  5.851e-03  -5.786 9.85e-09 ***
## I(time^2)    3.066e-04  3.852e-05   7.960 4.99e-15 ***
## I(time^3)   -8.833e-07  1.038e-07  -8.507  < 2e-16 ***
## I(time^4)    1.051e-09  1.218e-10   8.629  < 2e-16 ***
## I(time^5)   -4.468e-13  5.155e-14  -8.667  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.38 on 933 degrees of freedom
## Multiple R-squared:  0.2586, Adjusted R-squared:  0.2546
## F-statistic: 65.08 on 5 and 933 DF,  p-value: < 2.2e-16
```

```r
summary(fit_poly6)
```

```
##
## Call:
## lm(formula = trend ~ time + I(time^2) + I(time^3) + I(time^4) +
##     I(time^5) + I(time^6), data = dat.trend)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -2.2637 -1.1037 -0.2130  0.7824  3.8588
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  4.465e+00  3.151e-01  14.168  < 2e-16 ***
## time         2.738e-03  9.265e-03   0.296 0.767639
## I(time^2)   -8.141e-05  8.572e-05  -0.950 0.342497
## I(time^3)    7.656e-07  3.422e-07   2.237 0.025497 *
## I(time^4)   -2.237e-09  6.619e-10  -3.379 0.000757 ***
## I(time^5)    2.630e-12  6.113e-13   4.302 1.87e-05 ***
## I(time^6)   -1.091e-15  2.160e-16  -5.051 5.30e-07 ***
```

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.362 on 932 degrees of freedom
## Multiple R-squared:  0.2783, Adjusted R-squared:  0.2737
## F-statistic: 59.91 on 6 and 932 DF,  p-value: < 2.2e-16
```

```r
summary(fit_poly7)
```

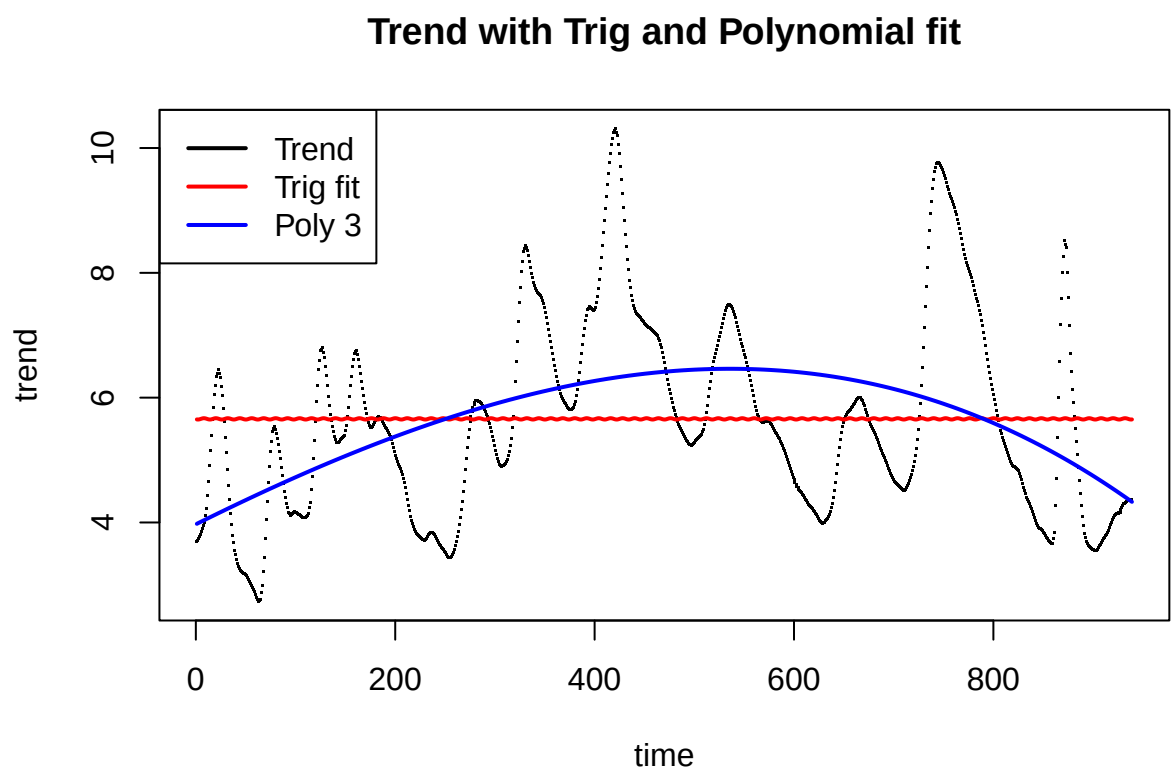
```
##
## Call:
## lm(formula = trend ~ time + I(time^2) + I(time^3) + I(time^4) +
##     I(time^5) + I(time^6) + I(time^7), data = dat.trend)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -2.4090 -1.0560 -0.1070  0.8746  3.4647
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  2.787e+00  3.440e-01   8.102 1.69e-15 ***
## time         1.013e-01  1.326e-02   7.637 5.53e-14 ***
## I(time^2)   -1.490e-03  1.635e-04  -9.117  < 2e-16 ***
## I(time^3)    9.075e-06  8.968e-07  10.120  < 2e-16 ***
## I(time^4)   -2.652e-08  2.522e-09 -10.516  < 2e-16 ***
## I(time^5)    3.982e-11  3.785e-12  10.521  < 2e-16 ***
## I(time^6)   -2.966e-14  2.880e-15 -10.298  < 2e-16 ***
## I(time^7)    8.683e-18  8.732e-19   9.944  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.296 on 931 degrees of freedom
## Multiple R-squared:  0.3476, Adjusted R-squared:  0.3427
## F-statistic: 70.87 on 7 and 931 DF,  p-value: < 2.2e-16
```

```r
summary(fit_poly8)
```

```
##
## Call:
## lm(formula = trend ~ time + I(time^2) + I(time^3) + I(time^4) +
##     I(time^5) + I(time^6) + I(time^7) + I(time^8), data = dat.trend)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -1.7865 -0.9008 -0.0317  0.6954  4.1089
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  5.097e+00  3.534e-01  14.425  < 2e-16 ***
## time        -7.246e-02  1.728e-02  -4.194 3.01e-05 ***
## I(time^2)    1.725e-03  2.731e-04   6.317 4.12e-10 ***
## I(time^3)   -1.594e-05  1.960e-06  -8.132 1.34e-15 ***
## I(time^4)    7.314e-08  7.463e-09   9.800  < 2e-16 ***
```

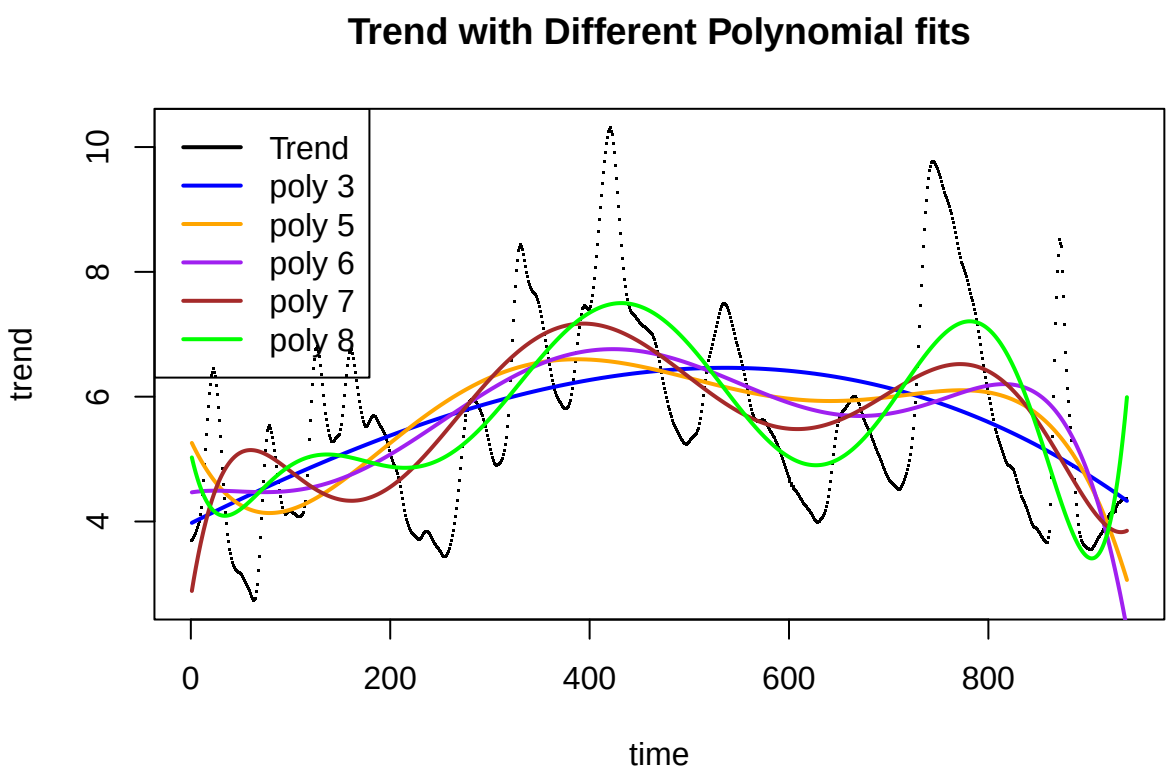
```
## I(time^5)   -1.805e-10  1.608e-11 -11.231  < 2e-16 ***
## I(time^6)    2.437e-13  1.966e-14  12.399  < 2e-16 ***
## I(time^7)   -1.694e-16  1.271e-17 -13.323  < 2e-16 ***
## I(time^8)    4.735e-20  3.374e-21  14.033  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.178 on 930 degrees of freedom
## Multiple R-squared:  0.4616, Adjusted R-squared:  0.457
## F-statistic: 99.68 on 8 and 930 DF,  p-value: < 2.2e-16
```

```r
# compare trig vs polynomial
plot(time, trend, pch=".", main="Trend with Trig and Polynomial fit")
lines(time, fitted(fit_trig), col="red", lwd=2)
lines(time, fitted(fit_poly3), col="blue", lwd=2)
legend("topleft",
       legend=c("Trend", "Trig fit","Poly 3"),
       col=c("black", "red", "blue"),
       lwd=2)
```




```r
# compare polynomial degrees
plot(time, trend, pch=".", main="Trend with Different Polynomial fits")

lines(time, fitted(fit_poly3), col="blue", lwd=2)
lines(time, fitted(fit_poly5), col="orange", lwd=2)
lines(time, fitted(fit_poly6), col="purple", lwd=2)
```

```r
lines(time, fitted(fit_poly7), col="brown", lwd=2)
lines(time, fitted(fit_poly8), col="green", lwd=2)

legend("topleft",
       legend=c("Trend", "poly 3","poly 5","poly 6","poly 7","poly 8"),
       col=c("black", "blue", "orange","purple","brown","green"),
       lwd=2)
```




```r
# AIC/BIC selection criteria
AIC(fit_poly3, fit_poly5, fit_poly6, fit_poly7, fit_poly8)
```

```
##           df      AIC
## fit_poly3  5 3345.936
## fit_poly5  7 3277.219
## fit_poly6  8 3253.863
## fit_poly7  9 3161.072
## fit_poly8 10 2982.724
```

```r
BIC(fit_poly3, fit_poly5, fit_poly6, fit_poly7, fit_poly8)
```

```
##           df      BIC
## fit_poly3  5 3370.161
## fit_poly5  7 3311.132
## fit_poly6  8 3292.622
```

```
## fit_poly7  9 3204.675
## fit_poly8 10 3031.172
```

The trig model for trend is not significant, with R^2 almost zero, which is expected.

AIC/BIC decrease as the polynomial degree increases, but the polynomial fit 8 seems to favor short term fluctuations rather than the overall structure. It is very unstable at the edge as well.

Seasonal model

```r
time <- 1:length(season)

# month index 1-12 repeating
month <- cycle(unratensa_ts_interp)

dat.season <- data.frame(
  season = as.numeric(season),
  time = time,
  month = as.numeric(month)
)

# trig fit, 1 harmonic
fit_season1 <- lm(season ~
                    cos(2*pi*month/12) + sin(2*pi*month/12),
                  data = dat.season)

# trig fit, 3 harmonics
fit_season2 <- lm(season ~
                    cos(2*pi*month/12) + sin(2*pi*month/12) +
                    cos(4*pi*month/12) + sin(4*pi*month/12) +
                    cos(6*pi*month/12) + sin(6*pi*month/12),
                  data = dat.season)

# check model summaries
summary(fit_season1)
```

```
##
## Call:
## lm(formula = season ~ cos(2 * pi * month/12) + sin(2 * pi * month/12),
##     data = dat.season)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.41997 -0.20964 -0.01918  0.29451  0.46633
##
## Coefficients:
##                          Estimate Std. Error t value Pr(>|t|)
## (Intercept)             0.0008124  0.0092394   0.088 0.929949
## cos(2 * pi * month/12) -0.0495083  0.0130733  -3.787 0.000162 ***
## sin(2 * pi * month/12)  0.3026840  0.0130595  23.177  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##
## Residual standard error: 0.2831 on 936 degrees of freedom
## Multiple R-squared:  0.3706, Adjusted R-squared:  0.3693
## F-statistic: 275.6 on 2 and 936 DF,  p-value: < 2.2e-16
```

```r
summary(fit_season2)
```

```
##
## Call:
## lm(formula = season ~ cos(2 * pi * month/12) + sin(2 * pi * month/12) +
##     cos(4 * pi * month/12) + sin(4 * pi * month/12) + cos(6 *
##     pi * month/12) + sin(6 * pi * month/12), data = dat.season)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.157399 -0.081057  0.004208  0.094241  0.170761
##
## Coefficients:
##                          Estimate Std. Error t value Pr(>|t|)
## (Intercept)             6.604e-05  3.211e-03   0.021 0.983597
## cos(2 * pi * month/12) -5.051e-02  4.544e-03 -11.117  < 2e-16 ***
## sin(2 * pi * month/12)  3.016e-01  4.539e-03  66.453  < 2e-16 ***
## cos(4 * pi * month/12)  3.671e-02  4.541e-03   8.084 1.94e-15 ***
## sin(4 * pi * month/12)  3.400e-01  4.541e-03  74.875  < 2e-16 ***
## cos(6 * pi * month/12) -1.525e-01  4.544e-03 -33.559  < 2e-16 ***
## sin(6 * pi * month/12) -1.513e-02  4.539e-03  -3.333 0.000894 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.0984 on 932 degrees of freedom
## Multiple R-squared:  0.9243, Adjusted R-squared:  0.9238
## F-statistic:  1897 on 6 and 932 DF,  p-value: < 2.2e-16
```

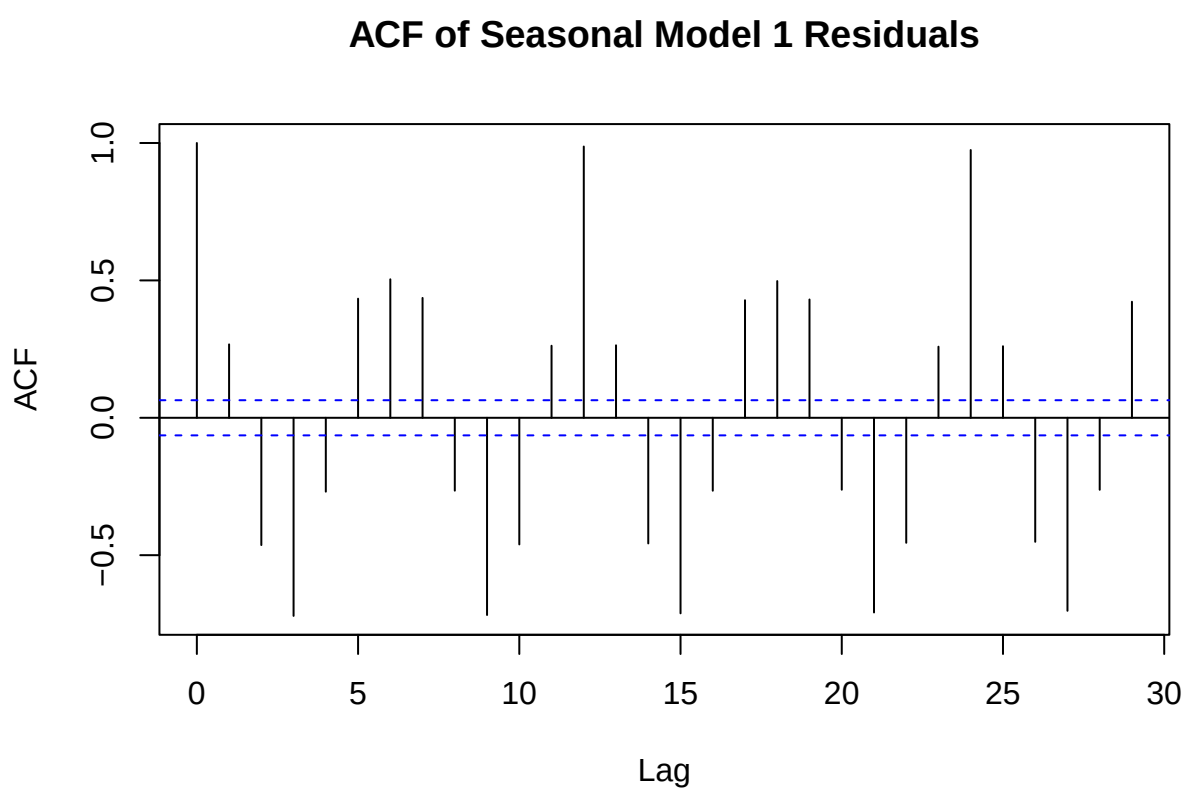
```r
# fitted seasonal values
season_fit1 <- fitted(fit_season1)
season_fit2 <- fitted(fit_season2)

# residuals from seasonal fits
season_resid1 <- residuals(fit_season1)
season_resid2 <- residuals(fit_season2)

# AIC/BIC comparison
AIC(fit_season1, fit_season2)
```

```
##             df        AIC
## fit_season1  4   299.9439
## fit_season2  8 -1680.7950
```

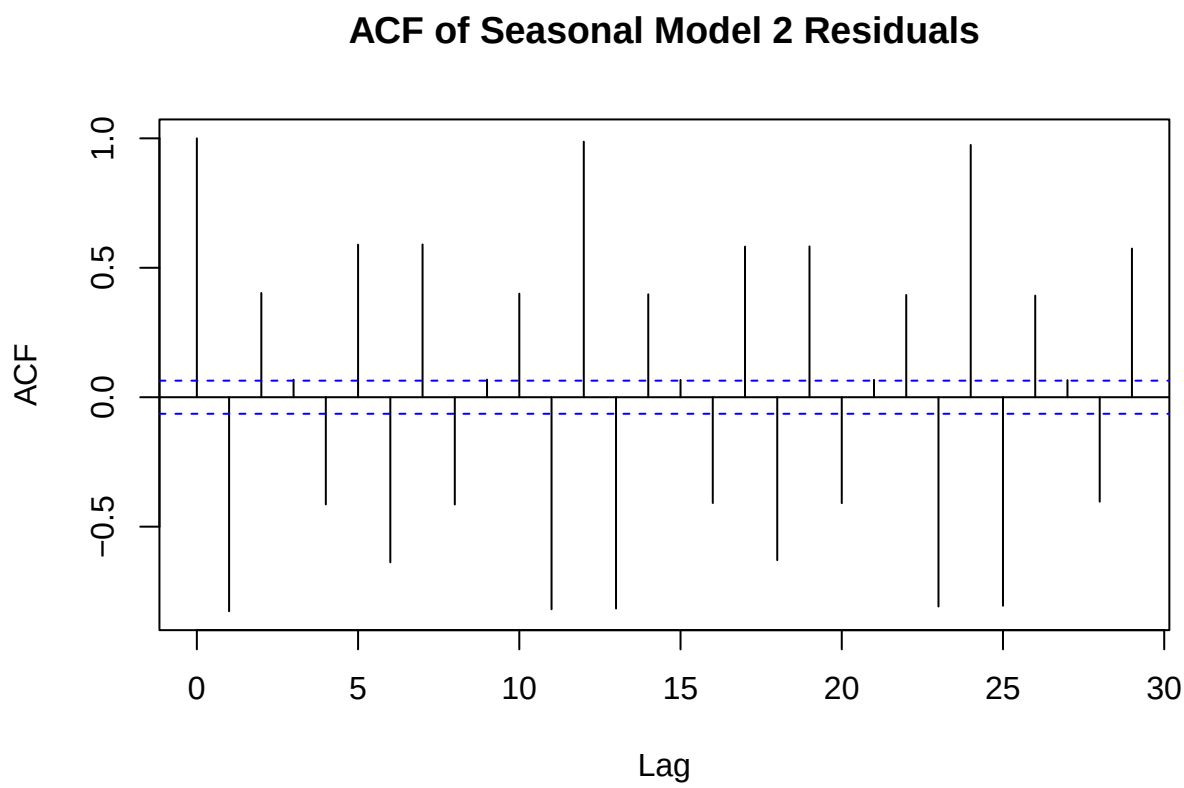
```r
BIC(fit_season1, fit_season2)
```

```
##             df        BIC
## fit_season1  4   319.3231
## fit_season2  8 -1642.0365
```

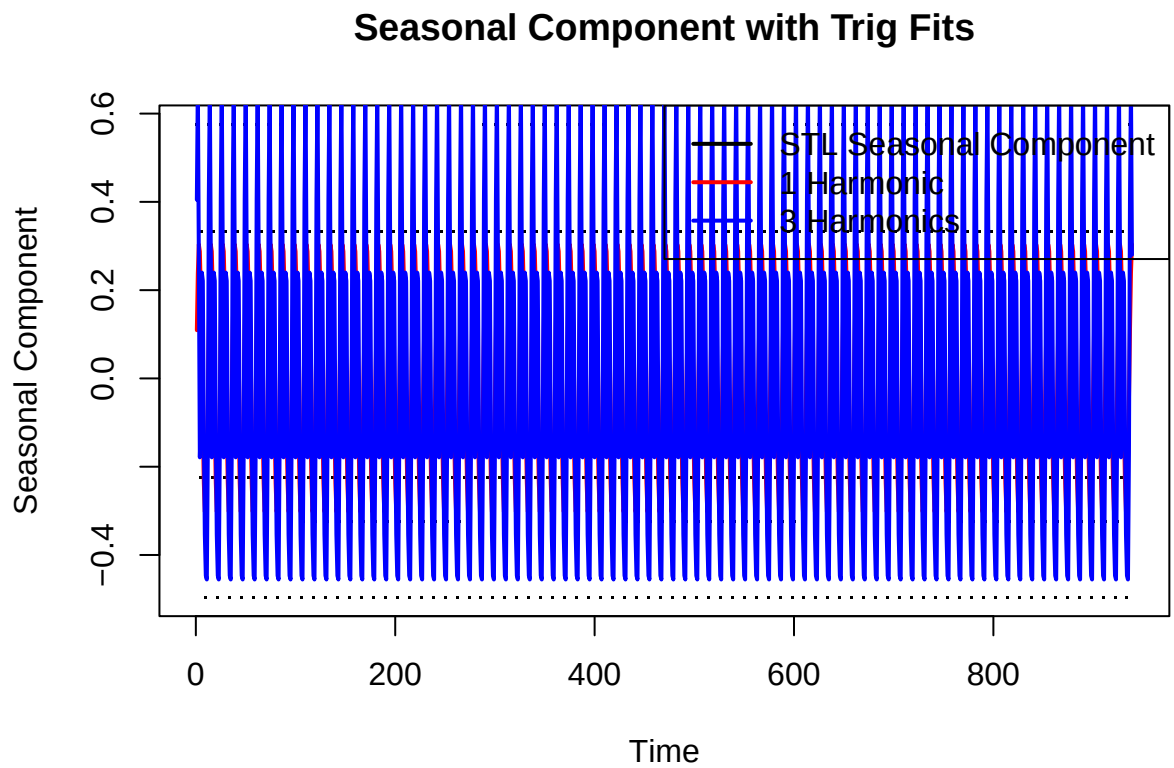
```r
# ACF checks
acf(season_resid1, main = "ACF of Seasonal Model 1 Residuals")
```



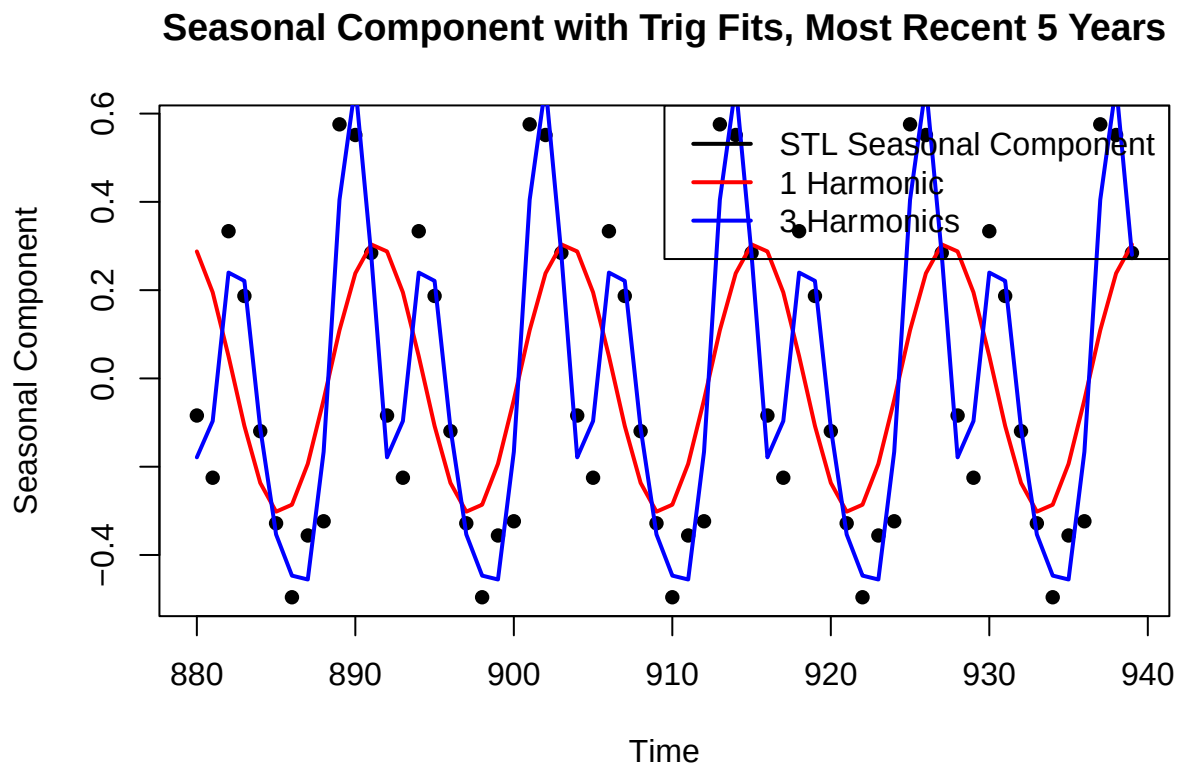

```r
acf(season_resid2, main = "ACF of Seasonal Model 2 Residuals")
```




```r
# full plot
plot(time, season, pch=".", col="black",
     main="Seasonal Component with Trig Fits",
     xlab="Time", ylab="Seasonal Component")

lines(time, season_fit1, col="red", lwd=2)
lines(time, season_fit2, col="blue", lwd=2)

legend("topright",
       legend=c("STL Seasonal Component", "1 Harmonic", "3 Harmonics"),
       col=c("black", "red", "blue"),
       lwd=2)
```




```r
# most recent 60 months / 5 years
n <- length(time)
idx <- (n-59):n

plot(time[idx], season[idx], pch=16, col="black",
     main="Seasonal Component with Trig Fits, Most Recent 5 Years",
     xlab="Time", ylab="Seasonal Component")

lines(time[idx], season_fit1[idx], col="red", lwd=2)
lines(time[idx], season_fit2[idx], col="blue", lwd=2)

legend("topright",
       legend=c("STL Seasonal Component", "1 Harmonic", "3 Harmonics"),
       col=c("black", "red", "blue"),
       lwd=2)
```

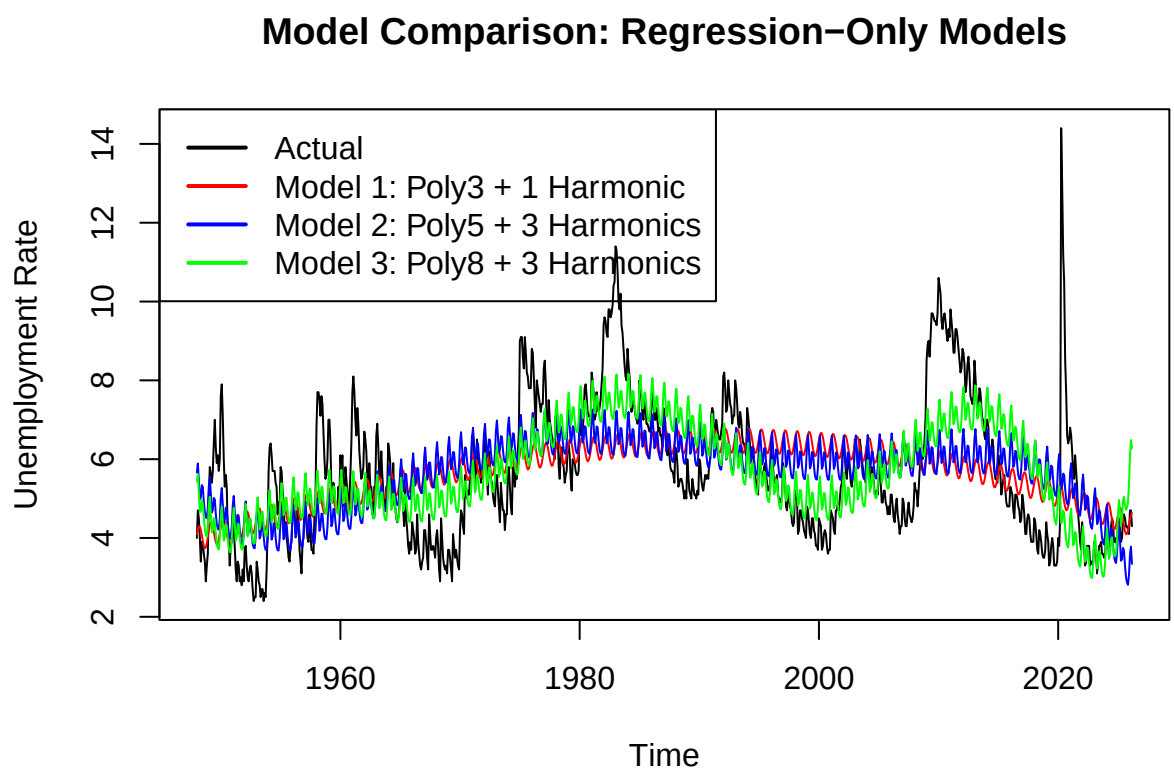


Combined Models

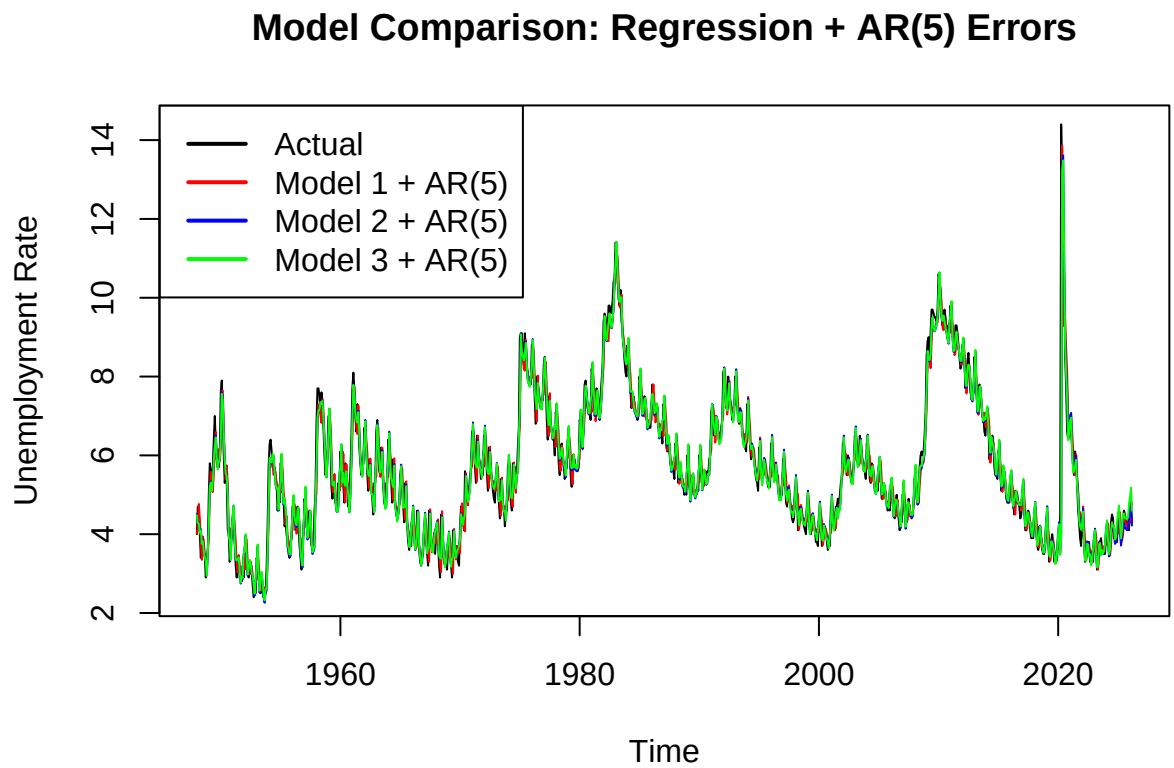
```
#trend comp
tr_poly3 <- fitted(fit_poly3)
tr_poly5 <- fitted(fit_poly5)
tr_poly8 <- fitted(fit_poly8)

#seasonal comp, monthly cycle, cycle() fixed to a 1-12 cycle
month <- cycle(unratensa_ts_interp)

# rebuild seasonal fits
coef_s1 <- coef(fit_season1)
coef_s2 <- coef(fit_season2)

ss1 <- coef_s1[1] +
       coef_s1[2]*cos(2*pi*month/12) +
       coef_s1[3]*sin(2*pi*month/12)

ss2 <- coef_s2[1] +
       coef_s2[2]*cos(2*pi*month/12) +
       coef_s2[3]*sin(2*pi*month/12) +
       coef_s2[4]*cos(4*pi*month/12) +
       coef_s2[5]*sin(4*pi*month/12) +
       coef_s2[6]*cos(6*pi*month/12) +
```

```r
       coef_s2[7]*sin(6*pi*month/12)

#combined regression-only models
yhat1 <- tr_poly3 + ss1                 # simple
yhat2 <- tr_poly5 + ss2                 # balanced
yhat3 <- tr_poly8 + ss2                 # flexible

# convert fitted values to time series
yhat1_ts <- ts(yhat1, start=start(unratensa_ts_interp), frequency=12)
yhat2_ts <- ts(yhat2, start=start(unratensa_ts_interp), frequency=12)
yhat3_ts <- ts(yhat3, start=start(unratensa_ts_interp), frequency=12)

# final residuals for regression-only models
final_resid1 <- unratensa_ts_interp - yhat1_ts
final_resid2 <- unratensa_ts_interp - yhat2_ts
final_resid3 <- unratensa_ts_interp - yhat3_ts

# Function to add AR(5) residual errors
add_ar5_errors <- function(yhat_ts, actual_ts) {
  resid <- actual_ts - yhat_ts

  ar5_fit <- arima(as.numeric(resid),
                   order = c(5, 0, 0),
                   include.mean = TRUE)

  ar5_resid <- residuals(ar5_fit)
  ar5_component <- as.numeric(resid) - ar5_resid

  yhat_ar5 <- as.numeric(yhat_ts) + ar5_component

  return(ts(yhat_ar5,
            start = start(actual_ts),
            frequency = frequency(actual_ts)))
}

# AR(5)-adjusted models
yhat1_ar5_ts <- add_ar5_errors(yhat1_ts, unratensa_ts_interp)
yhat2_ar5_ts <- add_ar5_errors(yhat2_ts, unratensa_ts_interp)
yhat3_ar5_ts <- add_ar5_errors(yhat3_ts, unratensa_ts_interp)

# final residuals for AR(5)-adjusted models
final_resid1_ar5 <- unratensa_ts_interp - yhat1_ar5_ts
final_resid2_ar5 <- unratensa_ts_interp - yhat2_ar5_ts
final_resid3_ar5 <- unratensa_ts_interp - yhat3_ar5_ts

#Regression-only models
plot(unratensa_ts_interp, col="black",
     main="Model Comparison: Regression-Only Models",
     ylab="Unemployment Rate")

lines(yhat1_ts, col="red")
lines(yhat2_ts, col="blue")
lines(yhat3_ts, col="green")
```

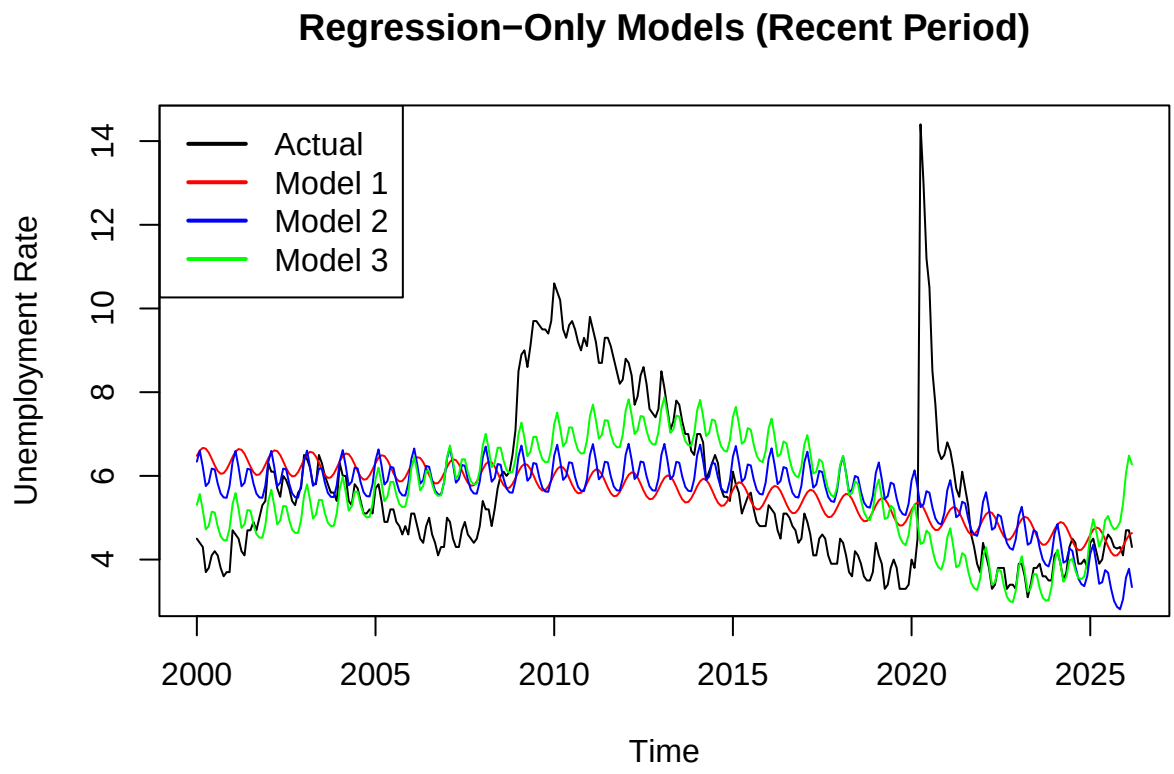
```r
legend("topleft",
       legend=c("Actual",
                "Model 1: Poly3 + 1 Harmonic",
                "Model 2: Poly5 + 3 Harmonics",
                "Model 3: Poly8 + 3 Harmonics"),
       col=c("black", "red", "blue", "green"),
       lwd=2)
```



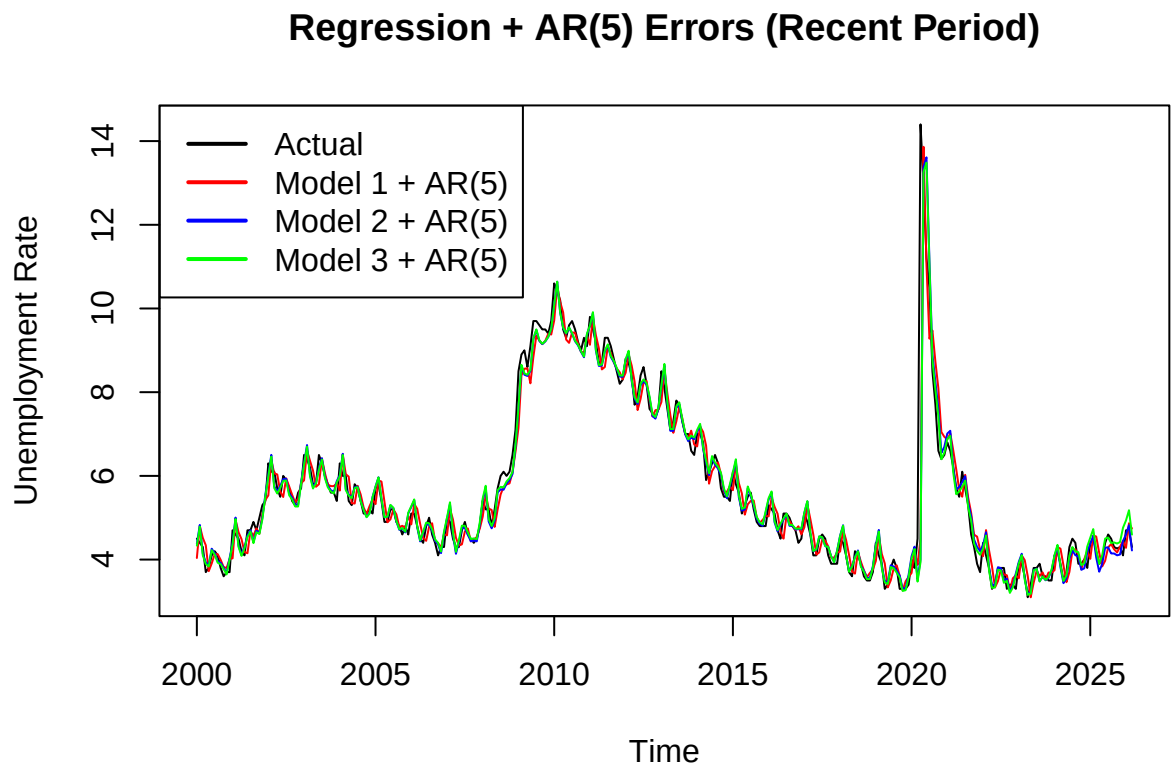

```r
#AR(5)-adjusted models
plot(unratensa_ts_interp, col="black",
     main="Model Comparison: Regression + AR(5) Errors",
     ylab="Unemployment Rate")

lines(yhat1_ar5_ts, col="red")
lines(yhat2_ar5_ts, col="blue")
lines(yhat3_ar5_ts, col="green")

legend("topleft",
       legend=c("Actual",
                "Model 1 + AR(5)",
                "Model 2 + AR(5)",
                "Model 3 + AR(5)"),
       col=c("black", "red", "blue", "green"),
       lwd=2)
```



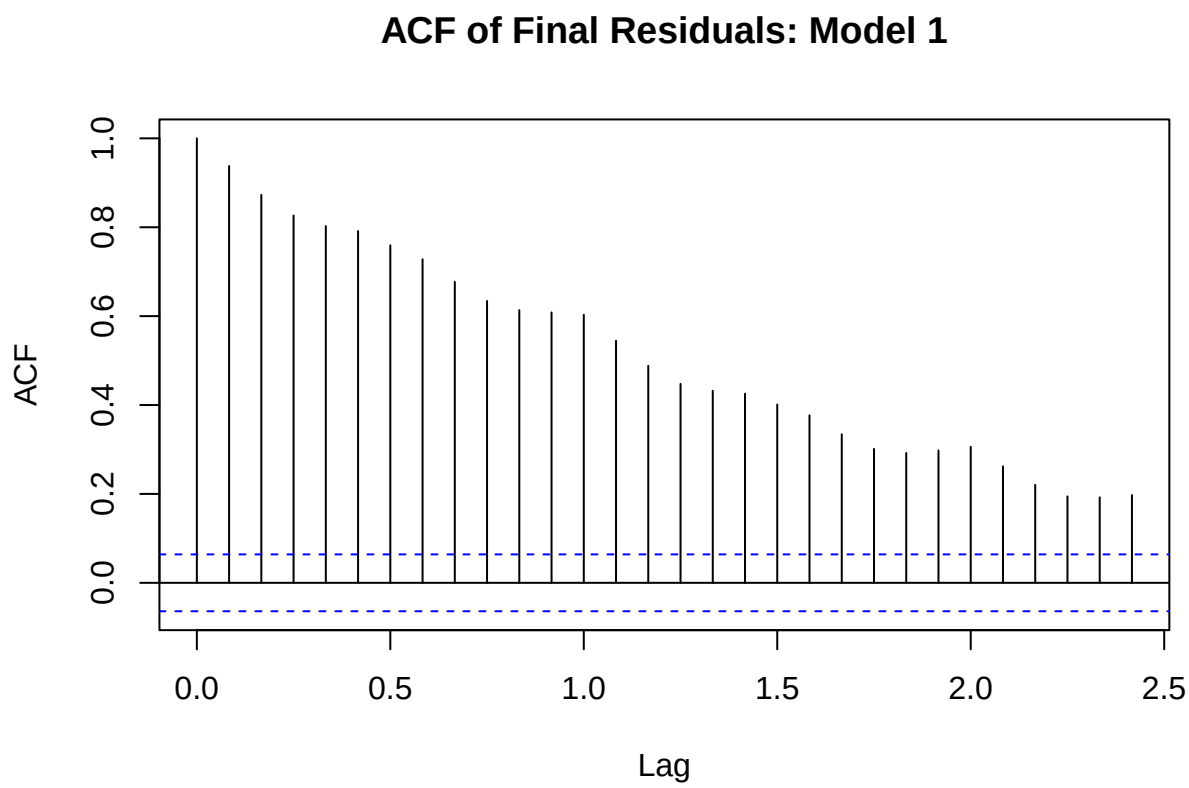

```r
# Recent period: regression-only models
start_plot <- c(2000,1)

plot(window(unratensa_ts_interp, start=start_plot), col="black",
     main="Regression-Only Models (Recent Period)",
     ylab="Unemployment Rate")

lines(window(yhat1_ts, start=start_plot), col="red")
lines(window(yhat2_ts, start=start_plot), col="blue")
lines(window(yhat3_ts, start=start_plot), col="green")

legend("topleft",
       legend=c("Actual",
                "Model 1",
                "Model 2",
                "Model 3"),
       col=c("black", "red", "blue", "green"),
       lwd=2)
```



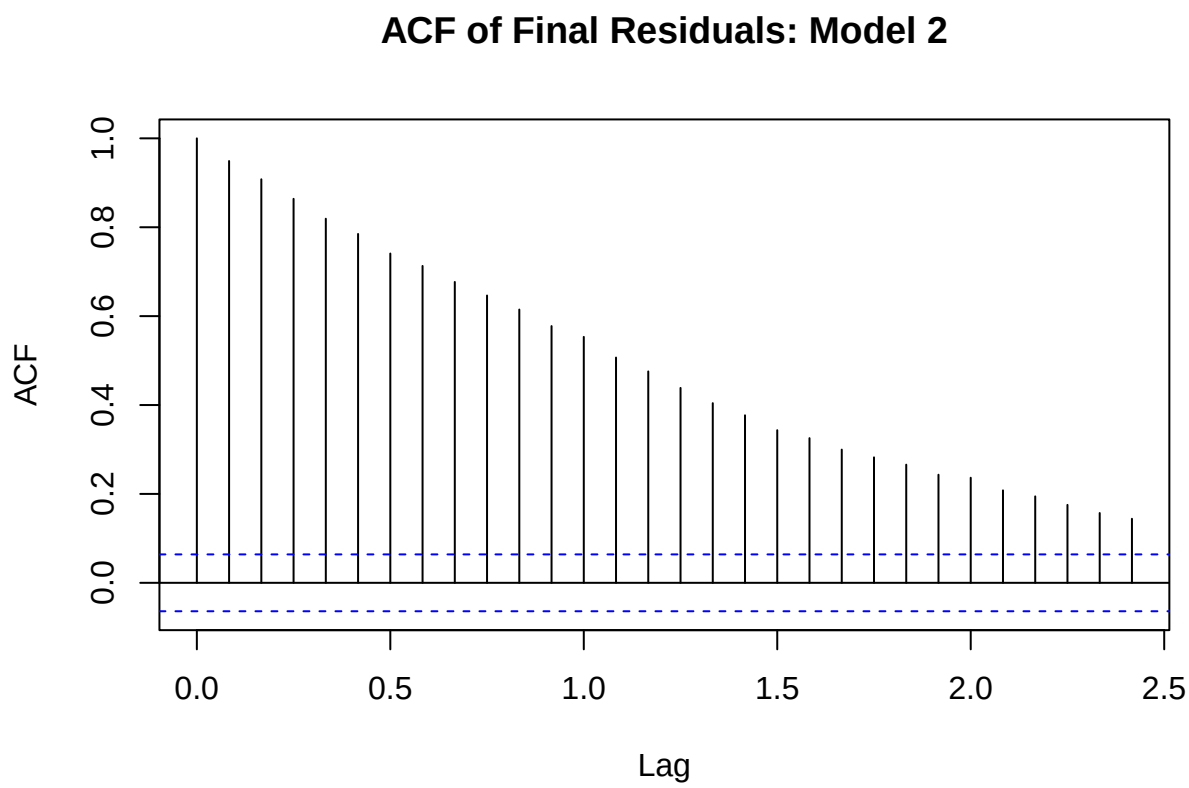

```r
# Recent period: AR(5)-adjusted models
plot(window(unratensa_ts_interp, start=start_plot), col="black",
     main="Regression + AR(5) Errors (Recent Period)",
     ylab="Unemployment Rate")

lines(window(yhat1_ar5_ts, start=start_plot), col="red")
lines(window(yhat2_ar5_ts, start=start_plot), col="blue")
lines(window(yhat3_ar5_ts, start=start_plot), col="green")

legend("topleft",
       legend=c("Actual",
                "Model 1 + AR(5)",
                "Model 2 + AR(5)",
                "Model 3 + AR(5)"),
       col=c("black", "red", "blue", "green"),
       lwd=2)
```



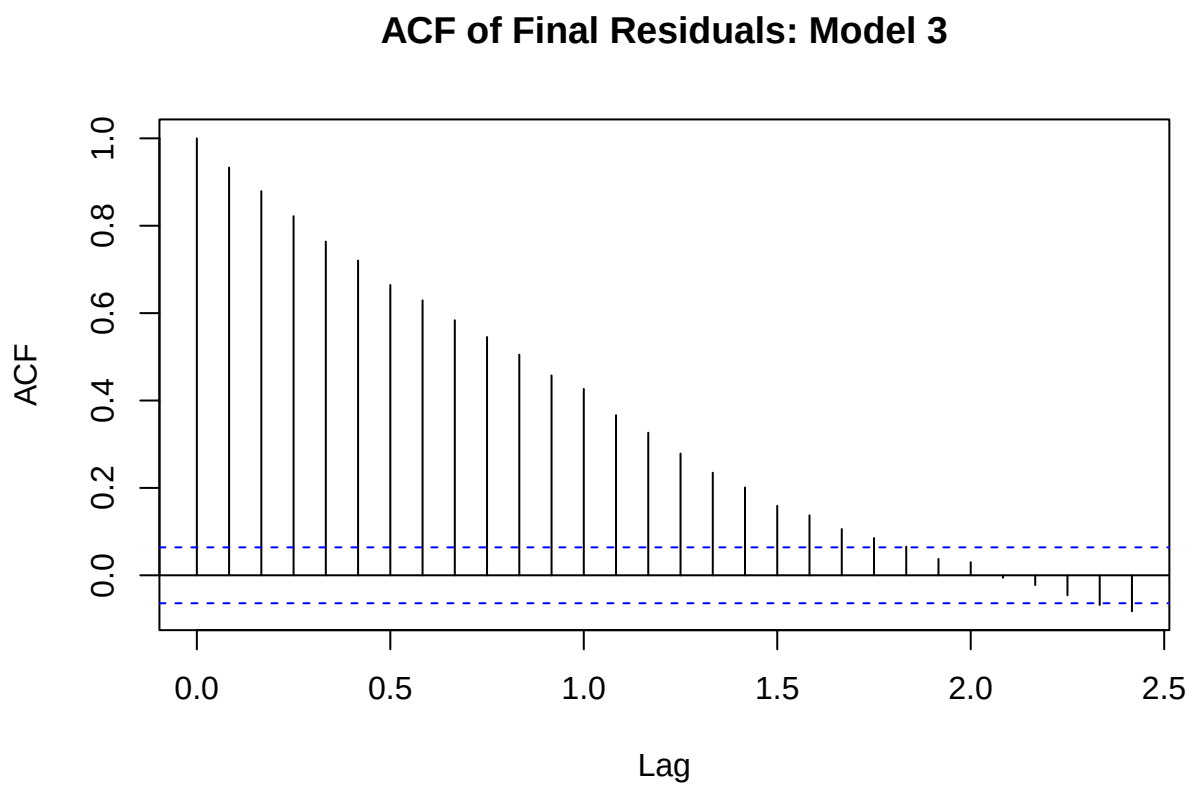

```r
# ACF comparisons
acf(final_resid1, main="ACF of Final Residuals: Model 1")
```



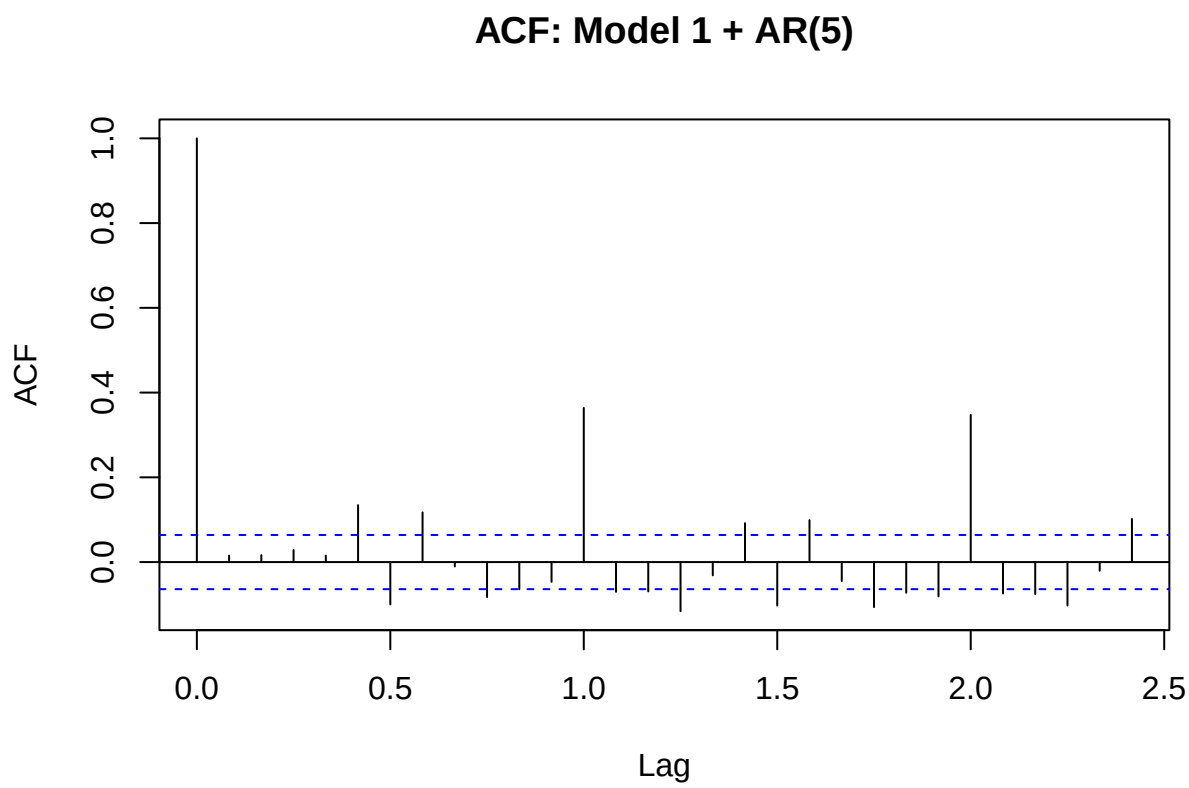

```
acf(final_resid2, main="ACF of Final Residuals: Model 2")
```



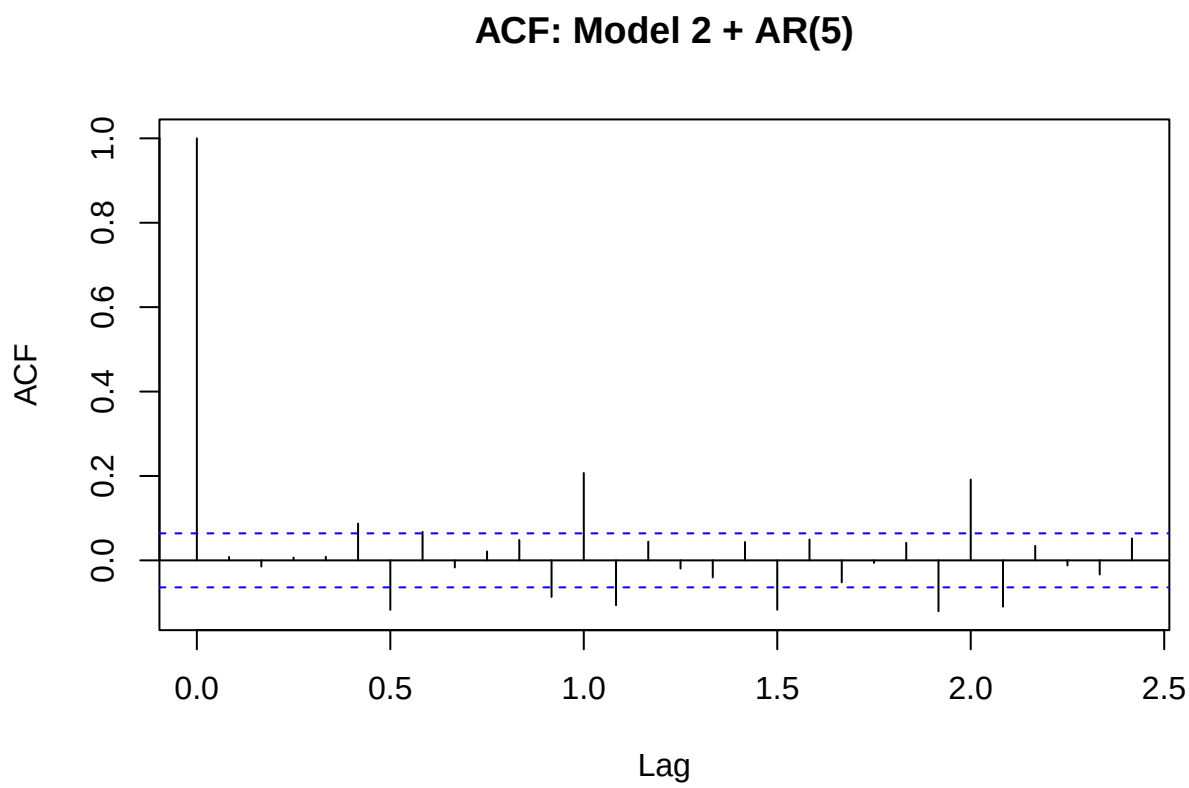

```
acf(final_resid3, main="ACF of Final Residuals: Model 3")
```



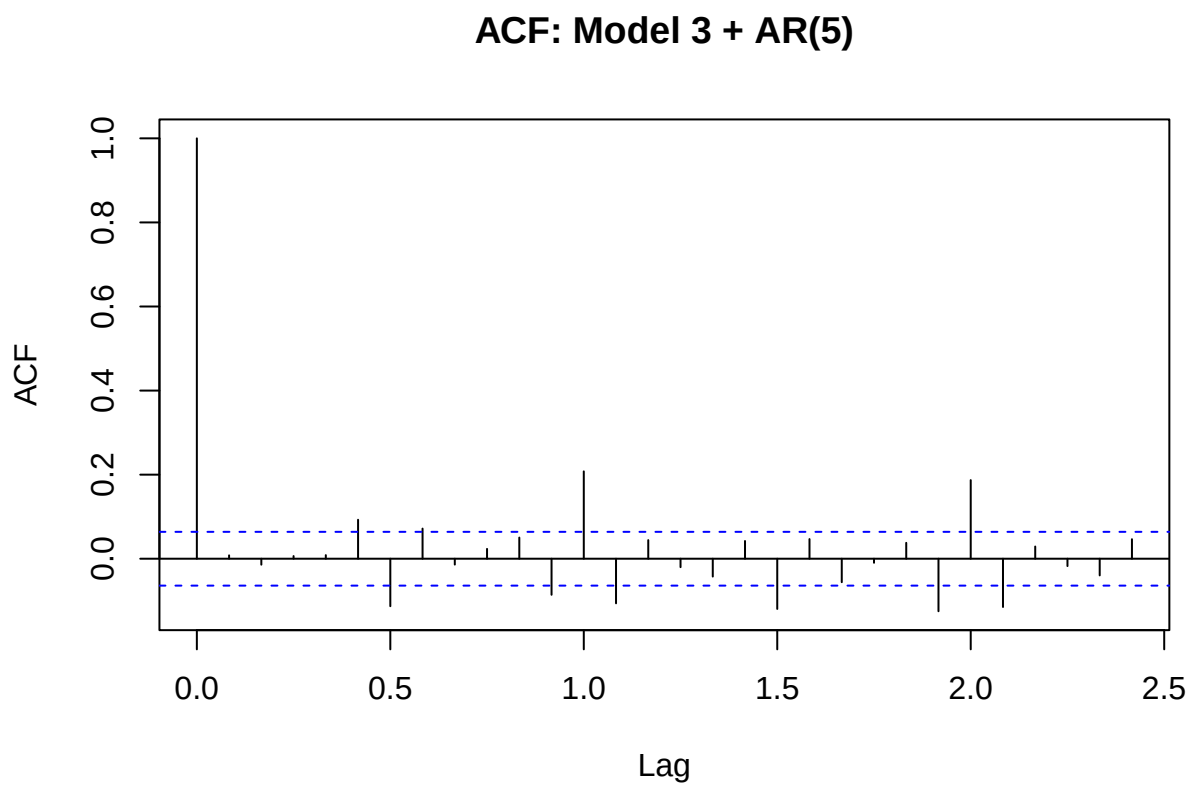

```r
acf(final_resid1_ar5, main="ACF: Model 1 + AR(5)")
```




```
acf(final_resid2_ar5, main="ACF: Model 2 + AR(5)")
```




```
acf(final_resid3_ar5, main="ACF: Model 3 + AR(5)")
```




```r
# Ljung-Box residual checks
Box.test(final_resid1, lag=12, type="Ljung-Box")
```

```
##
##  Box-Ljung test
##
## data:  final_resid1
## X-squared = 6316.1, df = 12, p-value < 2.2e-16
```

```r
Box.test(final_resid2, lag=12, type="Ljung-Box")
```

```
##
##  Box-Ljung test
##
## data:  final_resid2
## X-squared = 6350.2, df = 12, p-value < 2.2e-16
```

```r
Box.test(final_resid3, lag=12, type="Ljung-Box")
```

```
##
##  Box-Ljung test
##
## data:  final_resid3
## X-squared = 5243.4, df = 12, p-value < 2.2e-16
```

```r
Box.test(final_resid1_ar5, lag=12, type="Ljung-Box")
```

```
##
##  Box-Ljung test
##
## data:  final_resid1_ar5
## X-squared = 179.8, df = 12, p-value < 2.2e-16
```

```r
Box.test(final_resid2_ar5, lag=12, type="Ljung-Box")
```

```
##
##  Box-Ljung test
##
## data:  final_resid2_ar5
## X-squared = 75.741, df = 12, p-value = 2.661e-11
```

```r
Box.test(final_resid3_ar5, lag=12, type="Ljung-Box")
```

```
##
##  Box-Ljung test
##
## data:  final_resid3_ar5
## X-squared = 76.754, df = 12, p-value = 1.71e-11
```

The combined regression models demonstrate that incorporating both trend and seasonal components significantly improves the fit to the unemployment data. Among the regression-only models, the 8th-degree polynomial with 3 harmonics provides the closest visual fit, but this comes at the cost of overfitting and instability, particularly at the boundaries.

Residual diagnostics reveal that all regression-only models retain substantial autocorrelation, indicating that deterministic components alone are insufficient to capture the temporal dependence in the data. To address this, AR(5) models were applied to the residuals. This substantially reduced autocorrelation, as evidenced by the large decrease in Ljung-Box test statistics. However, the residuals of the AR(5)-adjusted models still exhibit statistically significant dependence.

Overall, while the combination of regression and AR(5) errors improves model fit, it does not fully resolve the time dependence in the series. This indicates that a more comprehensive stochastic modeling approach is required, motivating the use of ARIMA models in the next section.

II. Smoothing Methods & Forecasting

1. Naïve Baseline

The **seasonal naïve** method repeats the value from the same month one year prior — the simplest possible benchmark for a seasonal series:

$$\hat{X}_{t+h} = X_{t+h-m}, \quad m = 12$$

```r
naive_fit   <- snaive(unratensa_ts_interp, h = 24)

plot(naive_fit,
     main = "Seasonal Naïve Forecast - UNRATENSA",
     ylab = "Unemployment Rate (NSA)", xlab = "Time")

plot(residuals(naive_fit),
     main = "Residuals - Seasonal Naïve",
     ylab = "Residual", xlab = "Time")
abline(h = 0, col = "red", lty = 2)
```

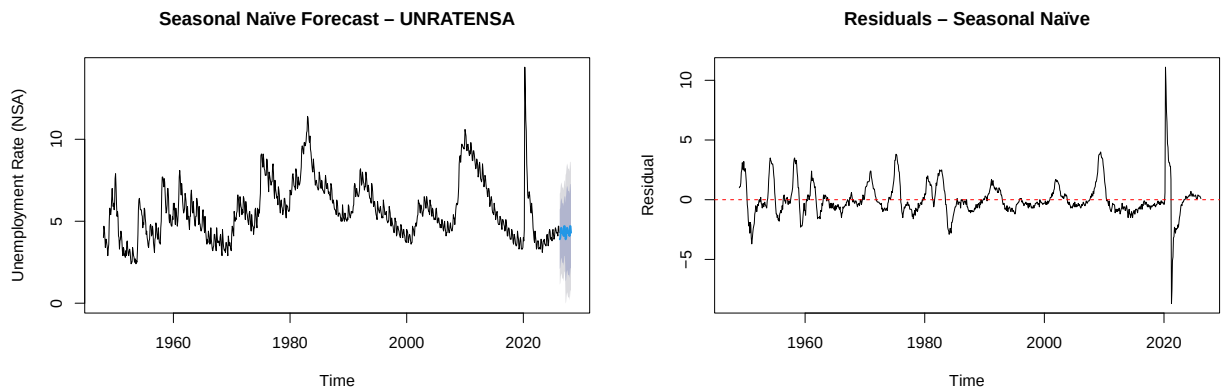


Figure 1: Seasonal naïve 24-month forecast (top) and its residuals (bottom)

The residuals show clear structure (large swings around recessions, COVID spike), confirming the naïve method leaves systematic patterns unexplained. It serves only as a reference benchmark.

---

2. Holt-Winters Exponential Smoothing

The Exploratory Data Analysis (EDA) and Seasonal and Trend decomposition using Loess (STL) decomposition established that UNRATENSA has both a time-varying trend and strong monthly seasonality. This rules out Simple Exponential Smoothing (level only) and Holt's method (trend, no seasonality) as appropriate models for the raw series. **Holt-Winters (HW)** is therefore the correct exponential smoothing model, estimating all three components simultaneously:

$$\ell_t = \alpha(X_t - S_{t-m}) + (1-\alpha)(\ell_{t-1} + b_{t-1})$$
$$b_t = \beta(\ell_t - \ell_{t-1}) + (1-\beta)\, b_{t-1}$$
$$S_t = \gamma(X_t - \ell_t) + (1-\gamma)\, S_{t-m}$$

where $m = 12$ for monthly data and $\alpha, \beta, \gamma \in (0, 1)$ control how quickly each component adapts to new observations.

```r
hw_fit <- HoltWinters(unratensa_ts_interp)
hw_fit   # display estimated alpha, beta, gamma
```

```
## Holt-Winters exponential smoothing with trend and additive seasonal component.
##
## Call:
## HoltWinters(x = unratensa_ts_interp)
##
## Smoothing parameters:
##  alpha: 0.9325619
##  beta : 0.01093134
##  gamma: 1
##
## Coefficients:
##             [,1]
## a    4.329301618
## b   -0.008964252
## s1   0.070343008
## s2  -0.194581134
## s3   0.063102535
## s4   0.150125618
## s5  -0.045341574
## s6  -0.353325427
## s7  -0.412697628
## s8  -0.443505478
## s9  -0.469169327
## s10  0.194788226
## s11  0.147037263
## s12 -0.029301618
```

```r
plot(unratensa_ts_interp, col = "black",
     main = "Holt-Winters - Fitted vs Observed",
     ylab = "Unemployment Rate (NSA)", xlab = "Time")
lines(hw_fit$fitted[, 1], col = "green", lwd = 1)
legend("topright", legend = c("Observed","HW Fitted"),
       col = c("black","green"), lwd = 2, cex = 0.8)
```

Result: $\hat{\alpha} \approx 0.933$, $\hat{\beta} \approx 0.011$, $\hat{\gamma} = 1.000$. The high α indicates the level estimate tracks recent observations closely; the low β means the trend slope changes slowly; $\gamma = 1$ means the seasonal factors are updated aggressively each month — appropriate for UNRATENSA's stable but precisely-timed seasonal pattern.

---

3. Forecast Results

```r
hw_forecast <- forecast(hw_fit, h = 24)

# Full history view
plot(hw_forecast,
     main = "HW 24-Month Forecast - UNRATENSA",
     ylab = "Unemployment Rate (NSA)", xlab = "Time")

# Zoomed view
recent_start <- c(2015, 1)
```

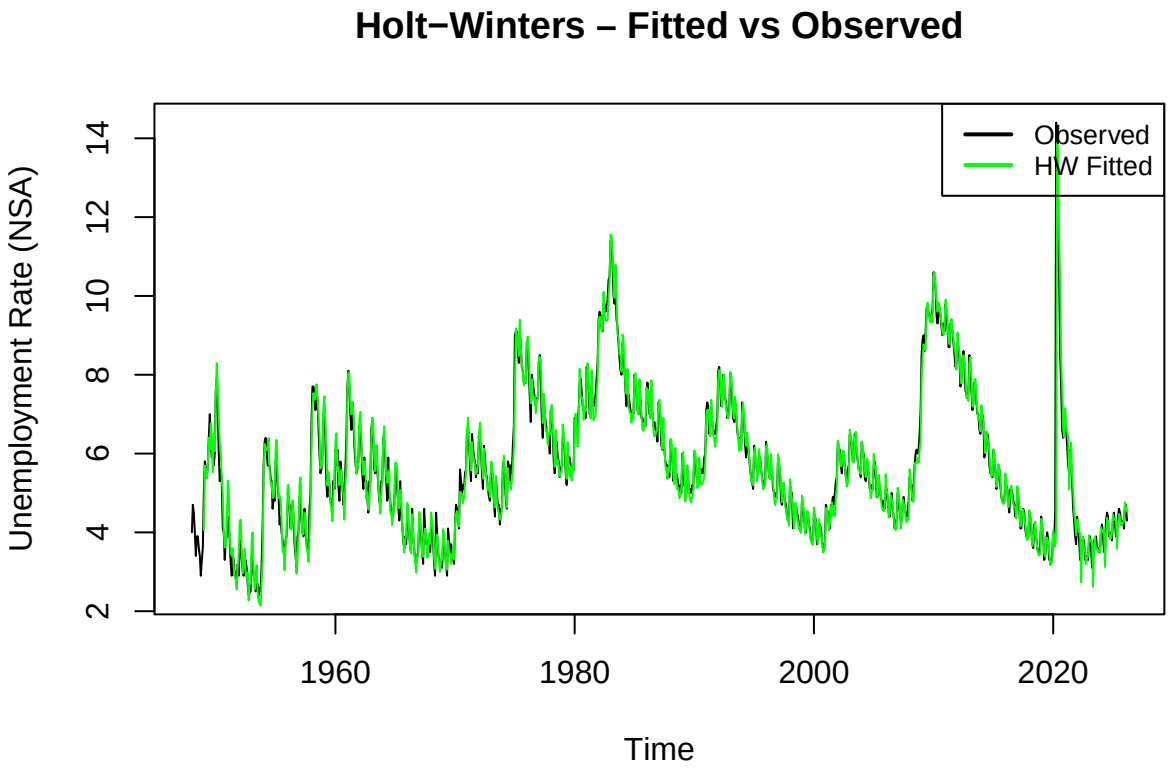


Figure 2: Holt-Winters fitted values overlaid on the full UNRATENSA series

```r
plot(window(unratensa_ts_interp, start = recent_start),
     xlim = c(2015, end(hw_forecast$mean)[1] + 1),
     ylim = range(c(window(unratensa_ts_interp, start = recent_start),
                    hw_forecast$lower, hw_forecast$upper), na.rm = TRUE),
     main = "HW Forecast - 2015 to 2028",
     ylab = "Unemployment Rate (NSA)", xlab = "Time", col = "black")
lines(hw_forecast$mean,        col = "darkgreen", lwd = 2)
lines(hw_forecast$lower[, 2],  col = "darkgreen", lty = 2)
lines(hw_forecast$upper[, 2],  col = "darkgreen", lty = 2)
legend("topright", legend = c("Observed","HW Forecast","95% CI"),
       col = c("black","darkgreen","darkgreen"),
       lty = c(1, 1, 2), lwd = 2, cex = 0.8)
```

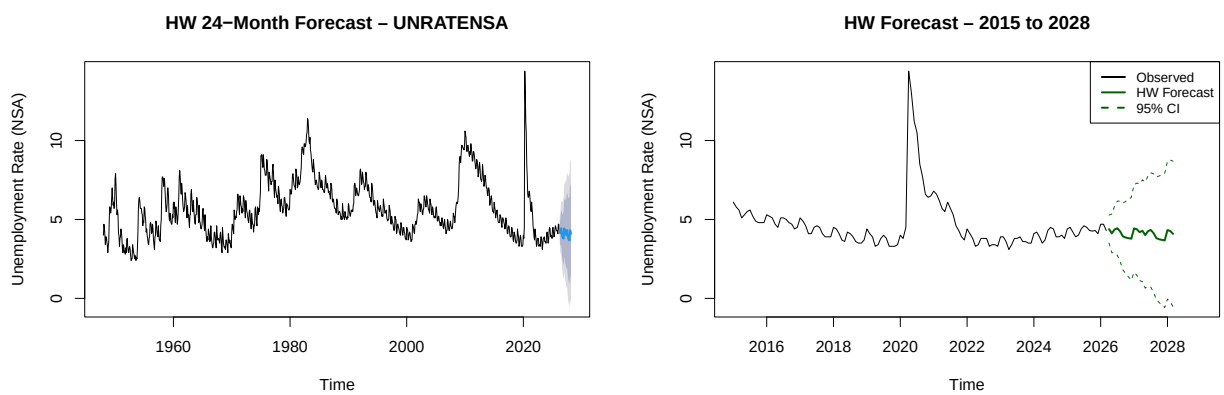


Figure 3: HW 24-month forecast (full history, top) and zoomed 2015–2028 view (bottom)

The model forecasts unemployment staying near **4.0–4.4%** through early 2028, with the characteristic seasonal dip in late summer/fall and uptick in winter–spring. Prediction intervals widen considerably beyond 12 months, reflecting genuine uncertainty over longer horizons.

---

4. Accuracy & Residual Diagnostics

```r
# Accuracy comparison across methods
cat("=== Seasonal Naïve ===\n");   print(accuracy(naive_fit))
```

```
## === Seasonal Naïve ===

##                       ME     RMSE      MAE       MPE    MAPE MASE      ACF1
## Training set 0.007403186 1.415716 0.931675 -2.442651 15.9323    1 0.9118357
```

```r
cat("\n=== Holt-Winters ===\n");    print(accuracy(hw_forecast))
```

```
##
## === Holt-Winters ===
```

```
##                       ME      RMSE       MAE        MPE     MAPE      MASE
## Training set -0.0211553 0.4543536 0.2263107 -0.4714777 4.134302 0.2429073
##                    ACF1
## Training set 0.1315664
```

```
hw_resid_ts <- ts(na.omit(residuals(hw_forecast)),
                  start = start(unratensa_ts_interp), frequency = 12)

plot(hw_resid_ts, main = "HW Residuals",
     ylab = "Residual", xlab = "Time")
abline(h = 0, col = "red", lty = 2)

acf(hw_resid_ts, lag.max = 36, main = "ACF - HW Residuals")
```

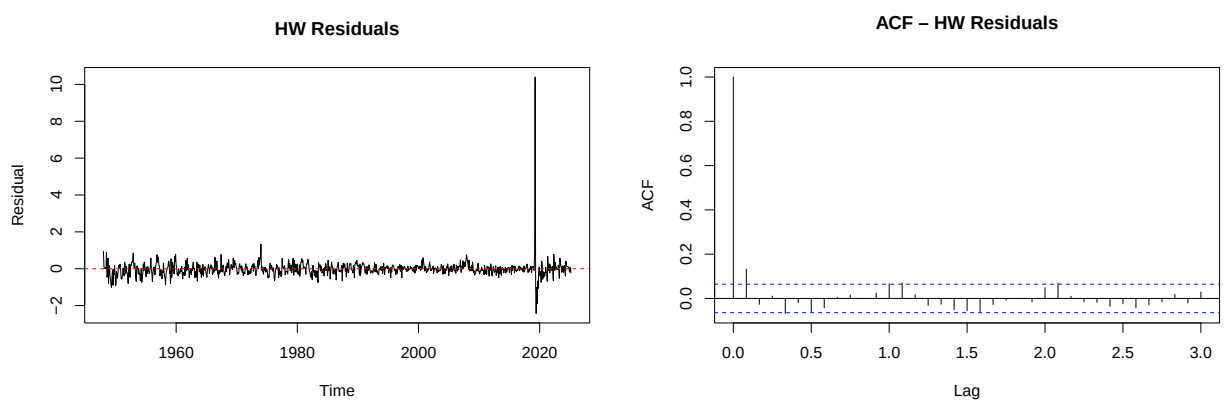


Figure 4: HW residual time plot (top) and ACF of residuals (bottom)

```
Box.test(hw_resid_ts, lag = 20, type = "Ljung-Box")
```

```
##
##  Box-Ljung test
##
## data:  hw_resid_ts
## X-squared = 47.617, df = 20, p-value = 0.0004814
```

Accuracy summary (training set):

Method	RMSE	MAE	MAPE
Seasonal Naïve	1.416	0.932	15.93%
Holt-Winters	**0.454**	**0.226**	**4.13%**

Holt-Winters achieves substantially lower error on all metrics. The residual ACF shows remaining autocorrelation (Ljung-Box $p < 0.001$), indicating the residuals are not white noise — this motivates the subsequent ARIMA modeling step to capture the remaining dependence structure.

Summary

Holt-Winters outperforms the naïve baseline by explicitly modeling UNRATENSA's trend and seasonality simultaneously. The estimated parameters ($\alpha \approx 0.93$, $\beta \approx 0.01$, $\gamma = 1$) are economically interpretable: the unemployment level shifts quickly, the trend evolves slowly, and the seasonal pattern is recalibrated every month. The residual diagnostics confirm that HW provides a meaningful improvement over naïve methods, while the remaining autocorrelation in residuals sets up the ARIMA analysis (Connor) to build on this foundation.